$ git log --oneline main

Du premier commit à la *prod*

01 **Ma première CI avec GitHub Actions** — TP1
Workflow Hello · ci.yml · échec volontaire · pull request · badge

02 **Une CI solide** — TP2
Lint · matrice · cache · artefact · protection de main · conflit · secret · pipelines à réparer

03 **Docker et livraison continue** — TP3
Dockerfile · Compose · GHCR · release v0.1.0 · staging & approbation · non-root

04 **GitOps avec Argo CD** — TP4
kind · Argo CD · tasks-config · replicas · self-heal · revert · git tag → pod

AUTEUR
Samir MAMA

DÉPÔT GITHUB (PUBLIC)
https://github.com/damesys2008-ship-it/tasks-api

DATE
Novembre 2026

$ git push origin tp1

Ma première CI *avec* GitHub Actions



OBJECTIF

Créer une CI à vérifier avec GitHub Actions, et ajouter un badge dans le README.

CAPTURES
7

DÉPÔT
damesys2008-ship-it/tasks-api

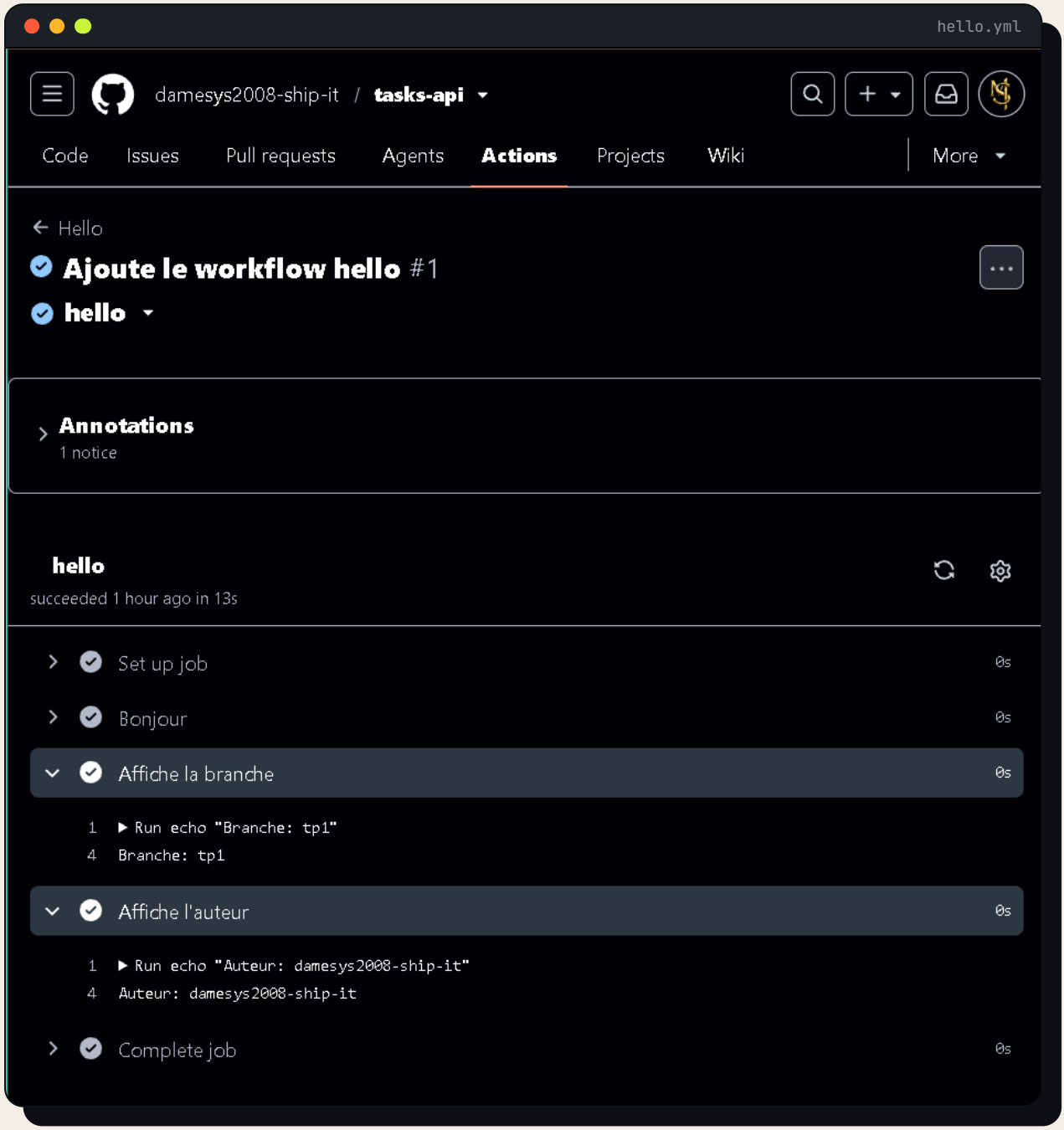
CI passing

TP 1-1

PASSED

Hello, world — version pipeline

Le run « *Hello* » réussi, avec les étapes `branche` et `auteur` dépliées.

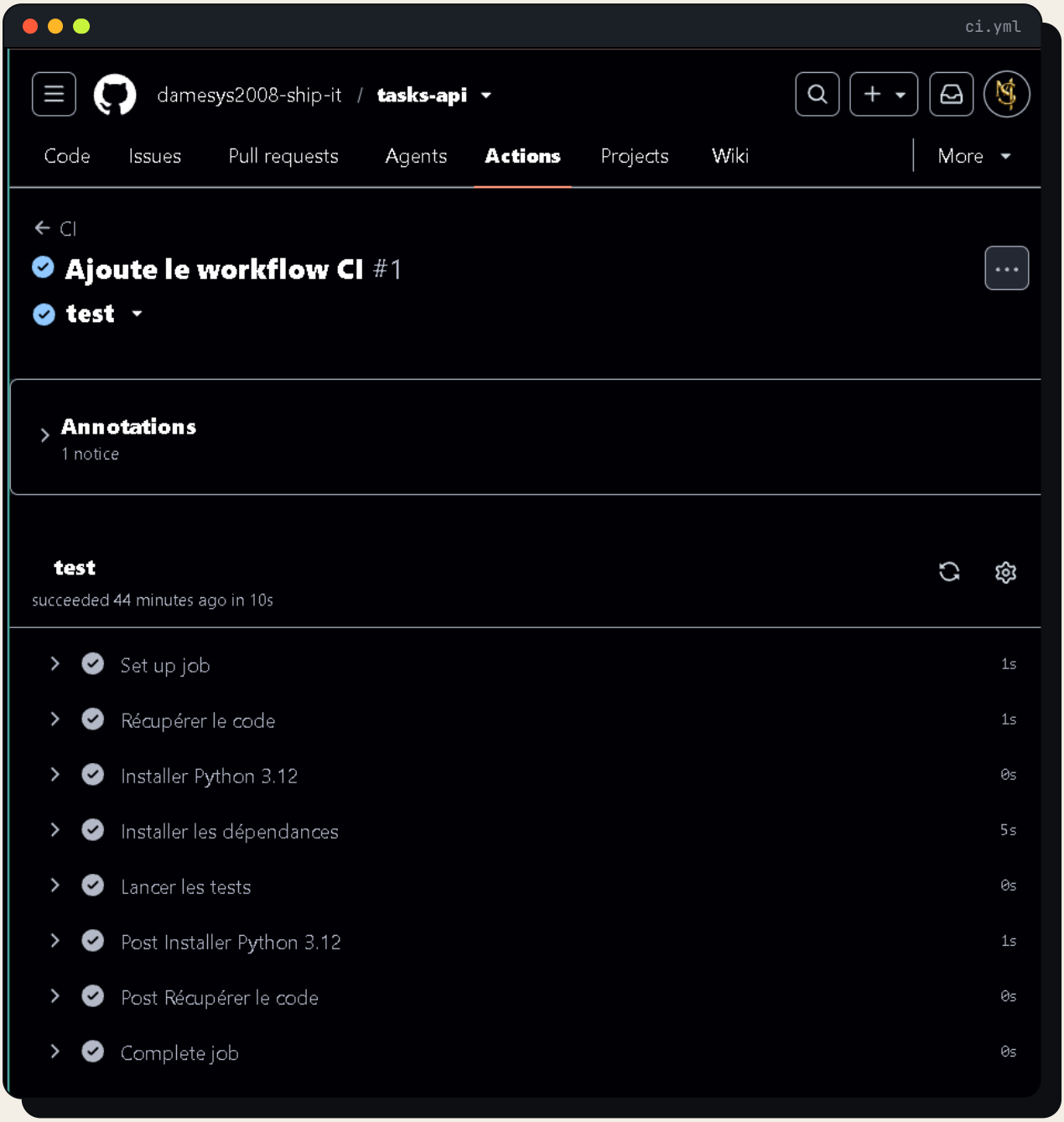


TP 1-2

PASSED

La vraie CI entre en scène

Le run `ci.yml` réussi, avec la liste complète des étapes.



TP 1-3

FAILED

On casse tout (exprès)

Provocation volontaire d'un échec de CI.

ci.yml

test

failed 47 minutes ago in 12s

Lancer les tests 0s

```
tests/test_api.py ...F.                                                  [ 50%]
tests/test_logic.py ..F..                                                [100%]

================================== FAILURES ===================================
_____________________________ test_add_and_list_tasks _____________________________

client = <FlaskClient <Flask 'app.main'>>

    def test_add_and_list_tasks(client):
        created = client.post("/tasks", json={"title": "  apprendre la ci "})
>       assert created.status_code == 201
E       assert 400 == 201
E        +  where 400 = <WrapperTestResponse streamed [400 BAD REQUEST]>.status_code

tests/test_api.py:29: AssertionError
____________________ test_is_valid_title_accepts_normal_title ____________________

    def test_is_valid_title_accepts_normal_title():
>       assert is_valid_title("Faire le TP1") is True
E       AssertionError: assert False is True
E        +  where False = is_valid_title('Faire le TP1')

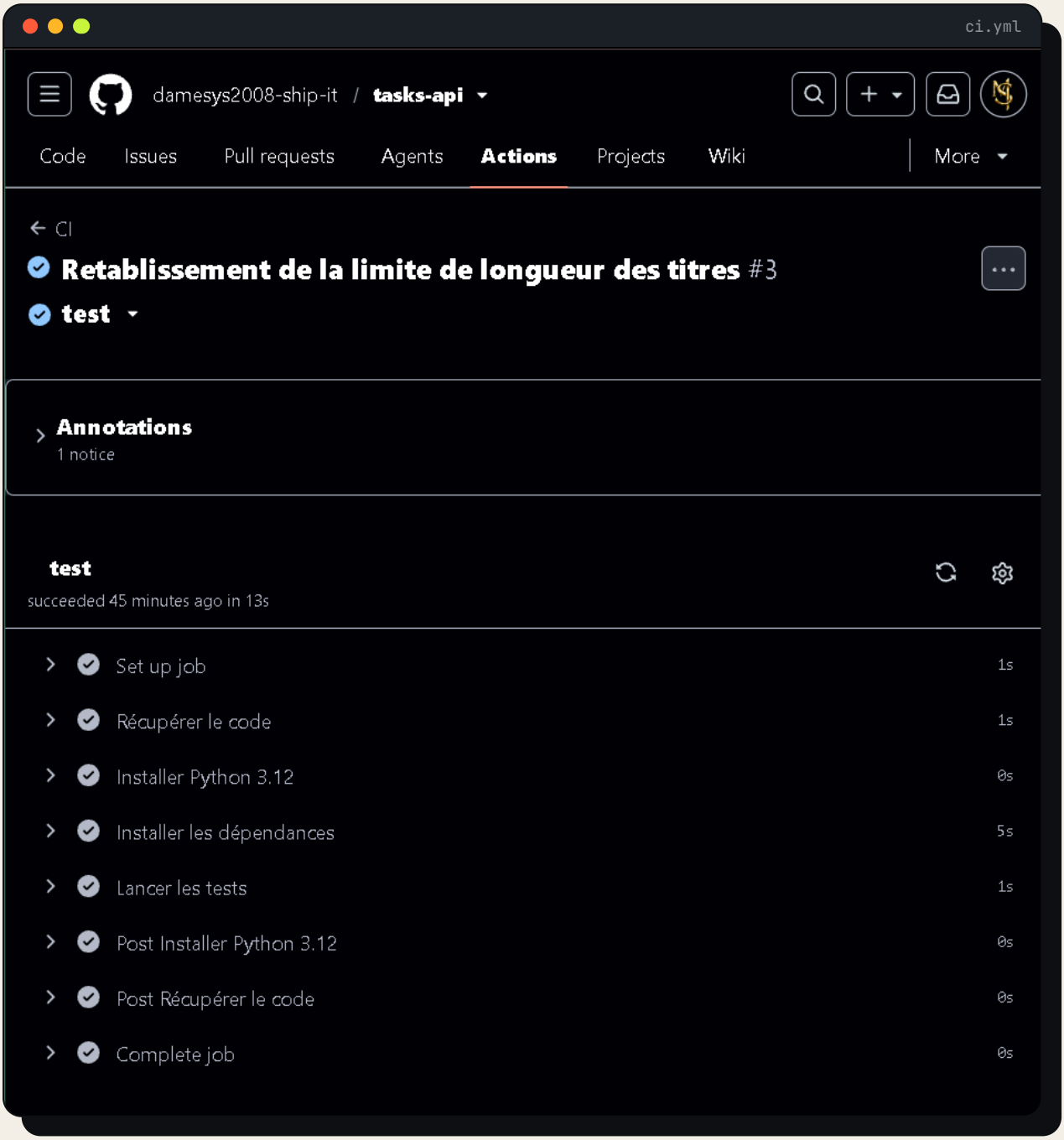
tests/test_logic.py:13: AssertionError
=========================== short test summary info ============================
FAILED tests/test_api.py::test_add_and_list_tasks - assert 400 == 201
 +  where 400 = <WrapperTestResponse streamed [400 BAD REQUEST]>.status_code
FAILED tests/test_logic.py::test_is_valid_title_accepts_normal_title - AssertionError: assert False is
True
 +  where False = is_valid_title('Faire le TP1')
======================== 2 failed, 8 passed in 0.20s ========================
Error: Process completed with exit code 1.
```

TP 1-4

PASSED

Retour au vert

Nouveau run après correction de l'échec.

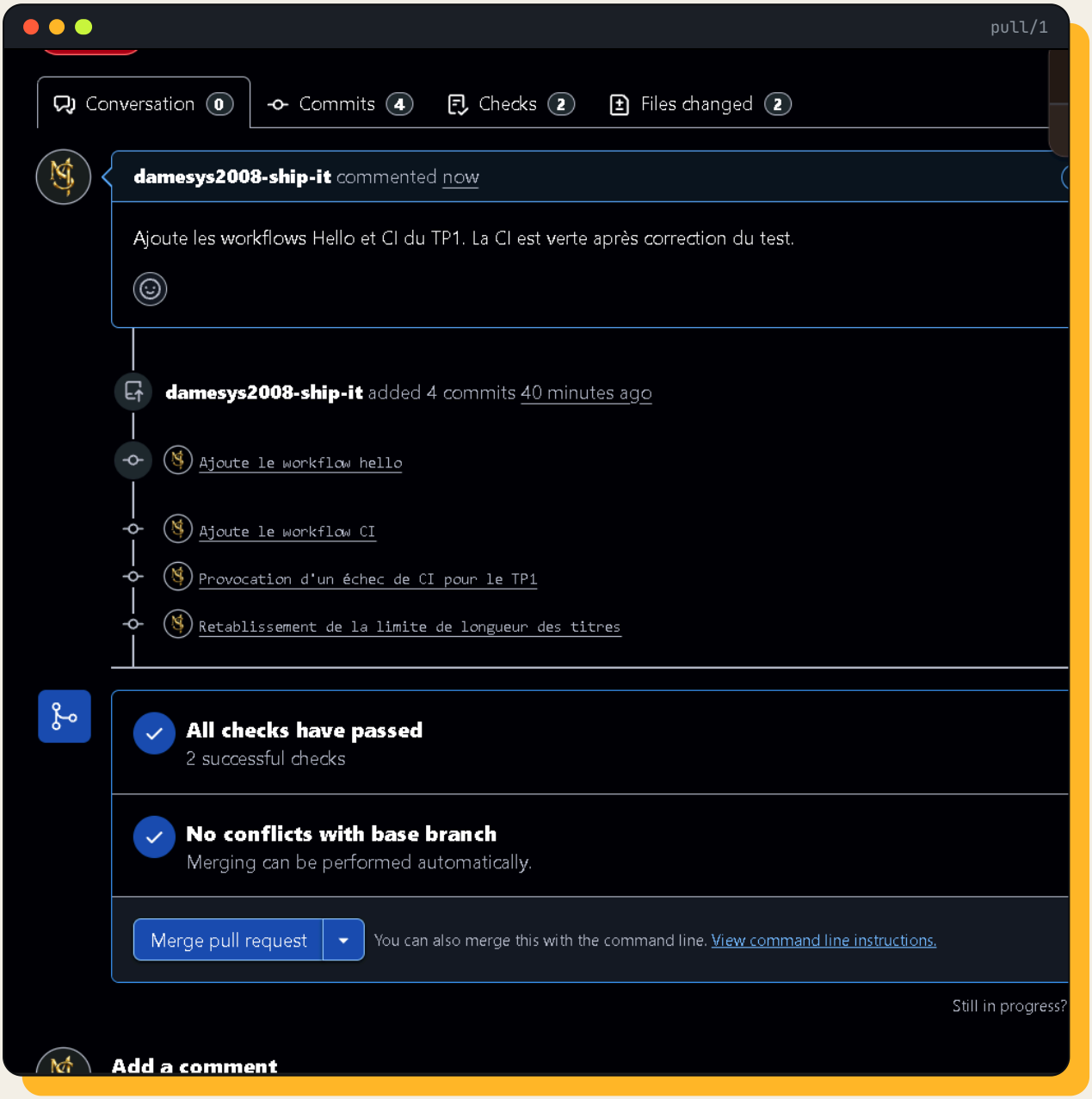


TP 1-5

CHECKS

Les checks font le videur

La pull request avec les vérifications avant la fusion.

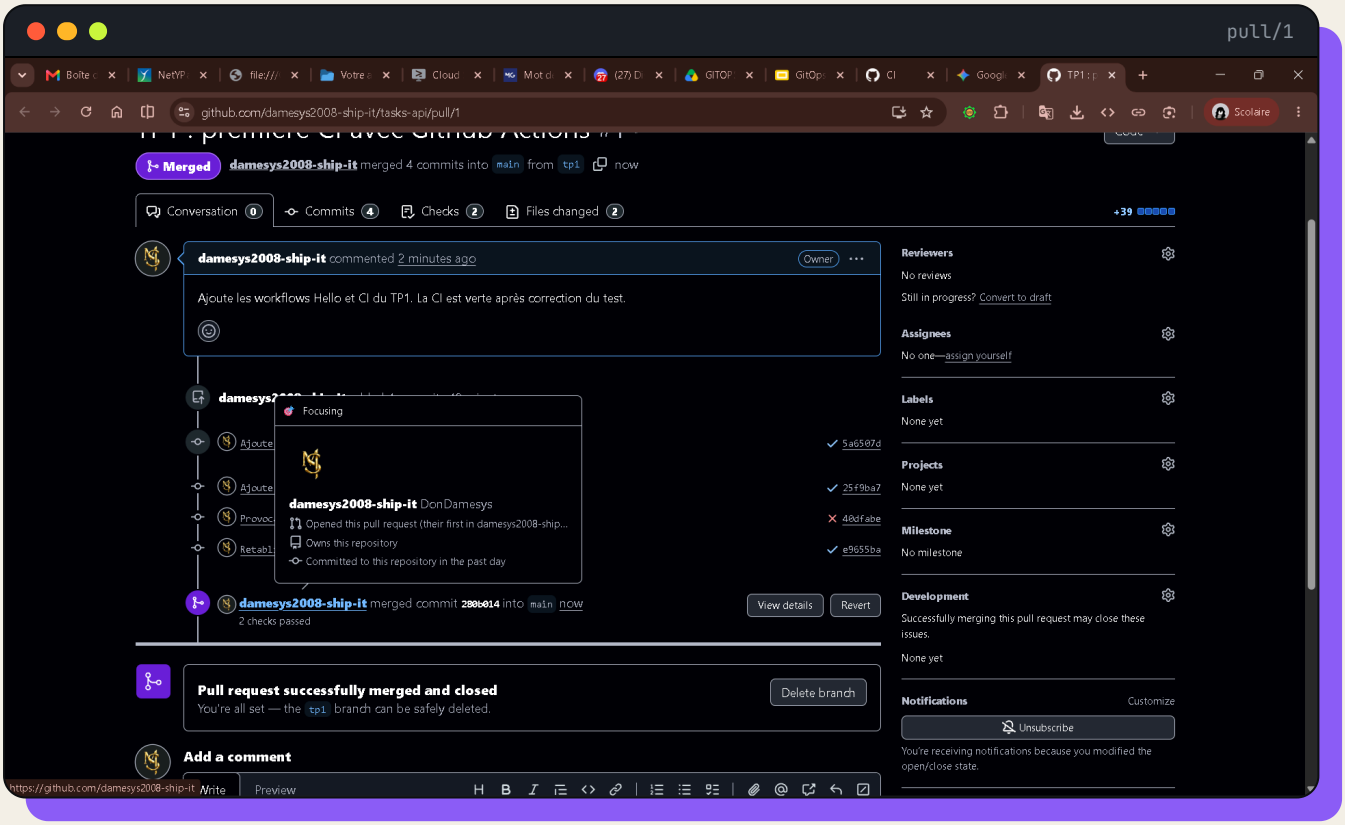


TP 1-6

MERGED

Fusion validée

La pull request après le merge dans `main`.



TP 1-7

PASSED

Le badge, la cerise

Le `README.md` sur GitHub avec le badge de CI affiché.



$ cat tp1/reponses.md

Réponses *aux questions — TP1*

01 Combien de temps a duré le run « Hello » ? Où voit-on les logs de chaque étape ?

Le run « Hello » a duré exactement **16 secondes**. Les logs, eux, se retrouvent en **ouvrant chaque étape** du job.

02 Quel test a échoué dans la partie 3, et à quelle ligne ? Qu'est-ce qui a permis de le trouver rapidement ?

Deux tests ont échoué : `test_add_and_list_tasks` et `test_is_valid_title_accepts_normal_title`. Le rapport de **pytest** affichait le nom des tests, les fichiers et les lignes, ce qui a permis de les retrouver rapidement.

03 Pourquoi attend-on que la CI soit verte avant de fusionner une pull request ?

Une CI verte avant la fusion confirme que les tests passent **sur une machine propre** et évite d'intégrer une modification qui casse le projet.

TP1 — all checks have passed · `exit 0`

02

$ git switch -c tp2

Une CI *vraiment* solide



OBJECTIF

Transformer la CI minimale du jour 1 en CI de qualité : analyse du code, couverture, plusieurs versions de Python, cache, protection de la branche `main`, travail à deux sur un même dépôt et gestion d'un secret.

CAPTURES
17

DÉPÔT
damesys2008-ship-it/tasks-api

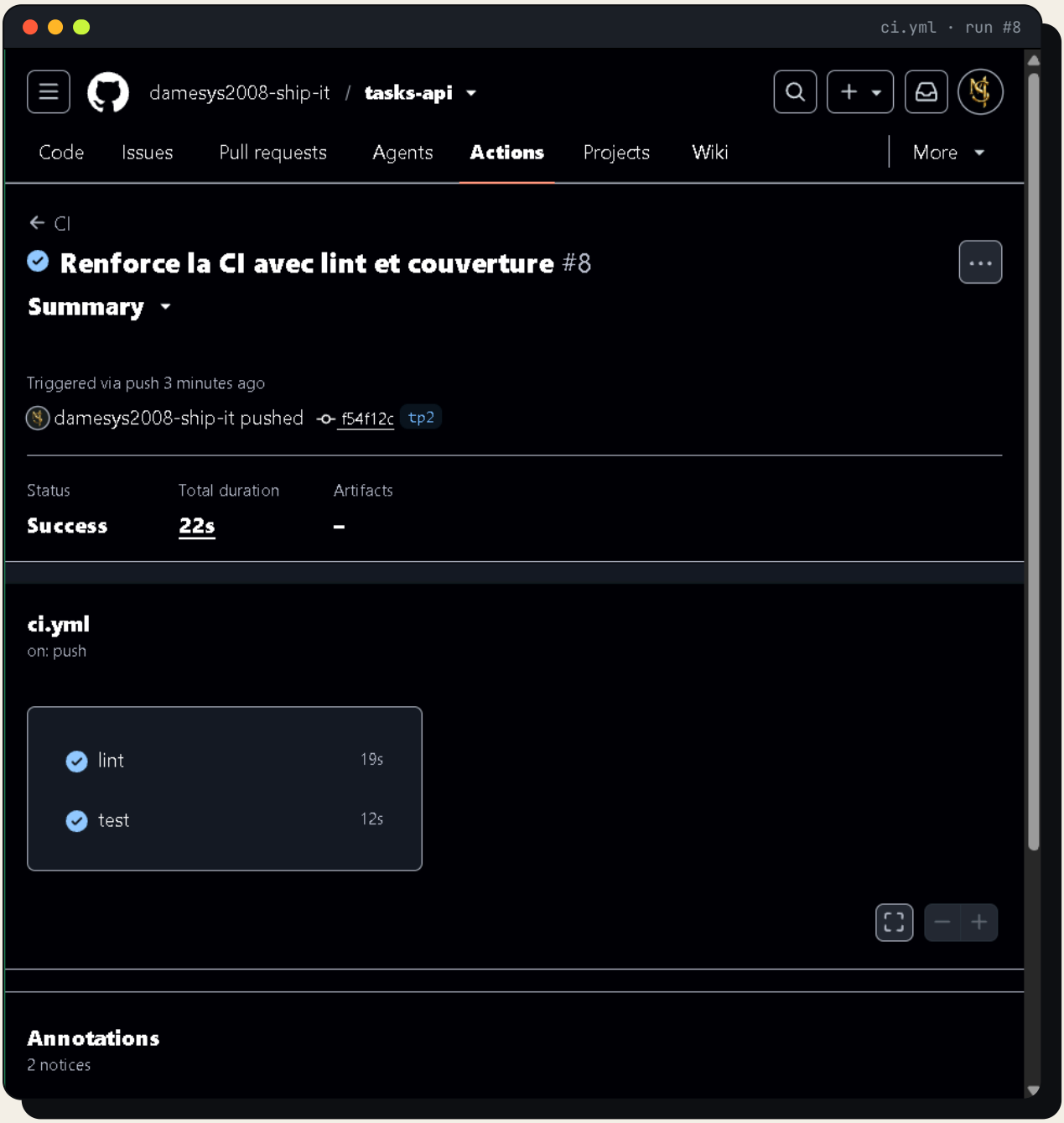
CI passing

TP 2-1

PASSED

Deux jobs valent mieux qu'un

Un run vert montrant les deux jobs `lint` et `test`.

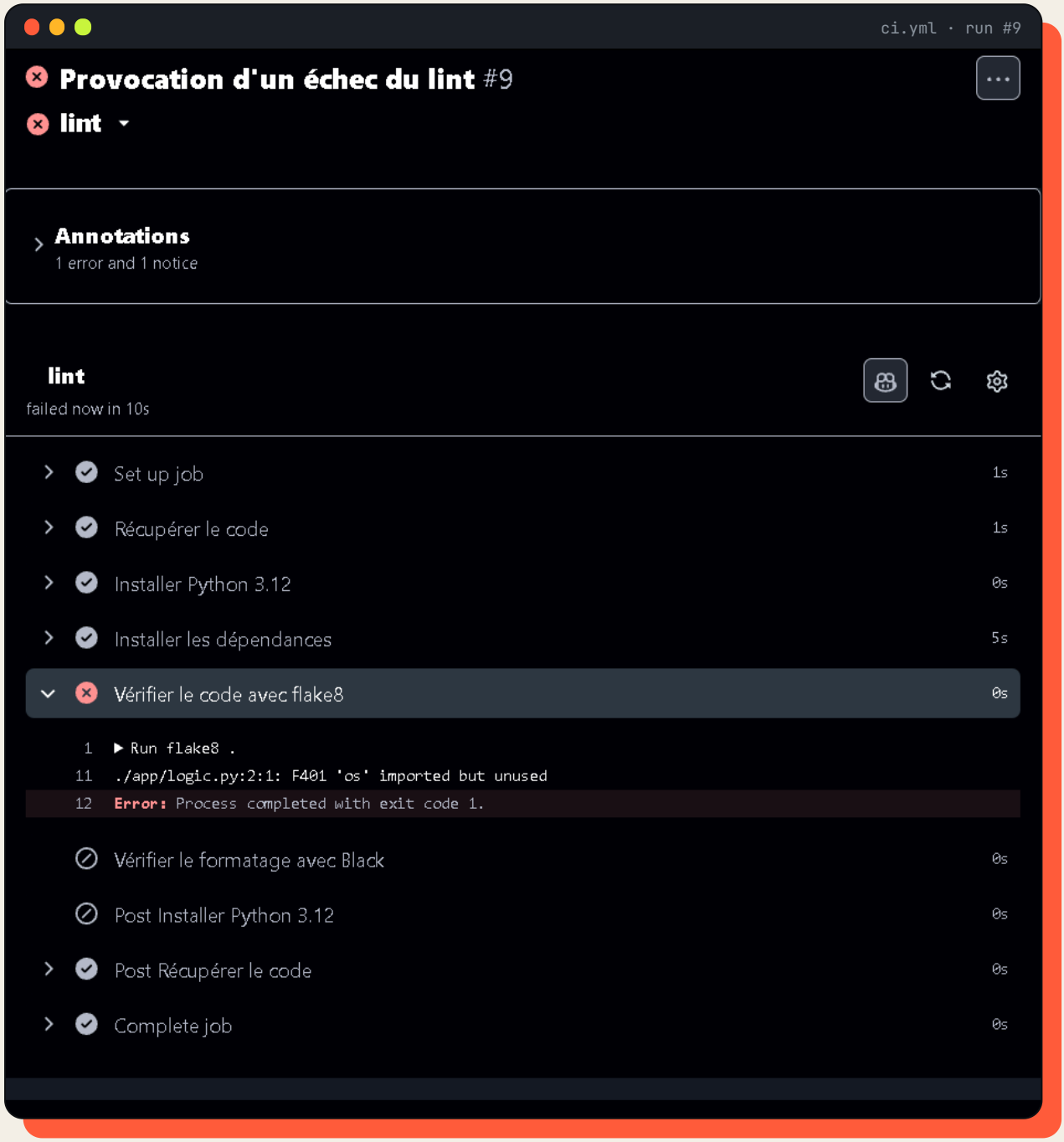


TP 2-2

FAILED

flake8 a l'œil

Le job `lint` rouge, avec le message de flake8 lisible : `F401 'os' imported but unused`.

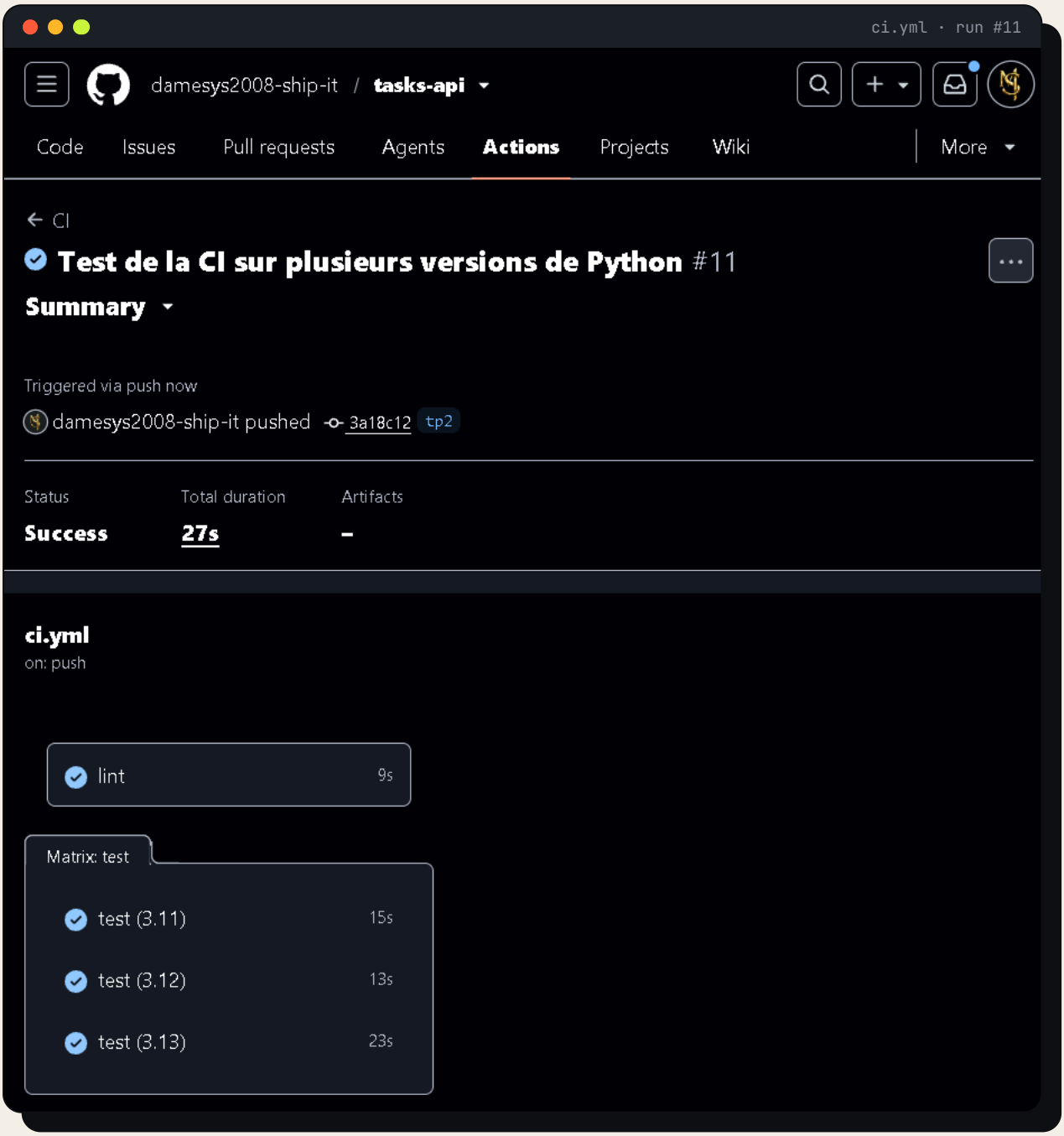


TP 2-3

MATRIX ×3

Trois Pythons, zéro jaloux

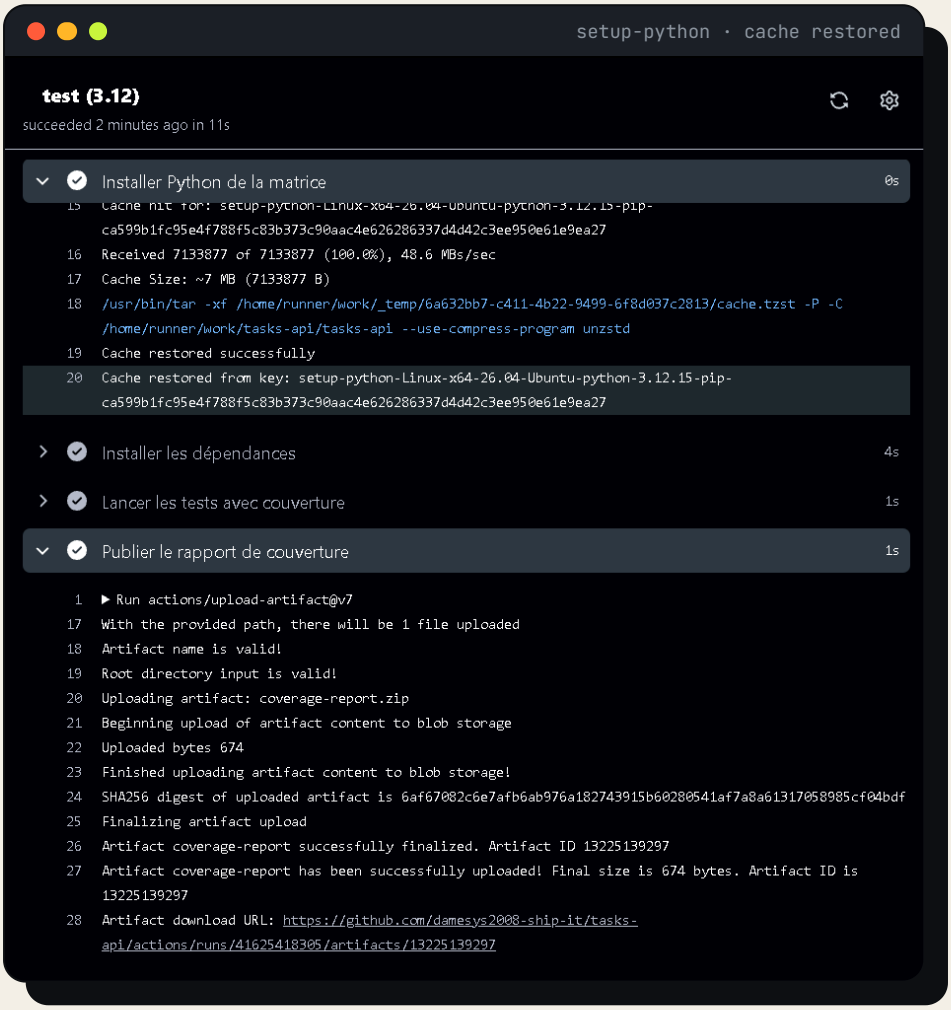
La matrice : `test (3.11)`, `test (3.12)` et `test (3.13)`.



CACHE + ARTEFACT

TP 2-4 Cache restauré, rapport emballé

L'étape setup-python du 2e run avec la mention du cache restauré, et la section « Artifacts » de la page du run.

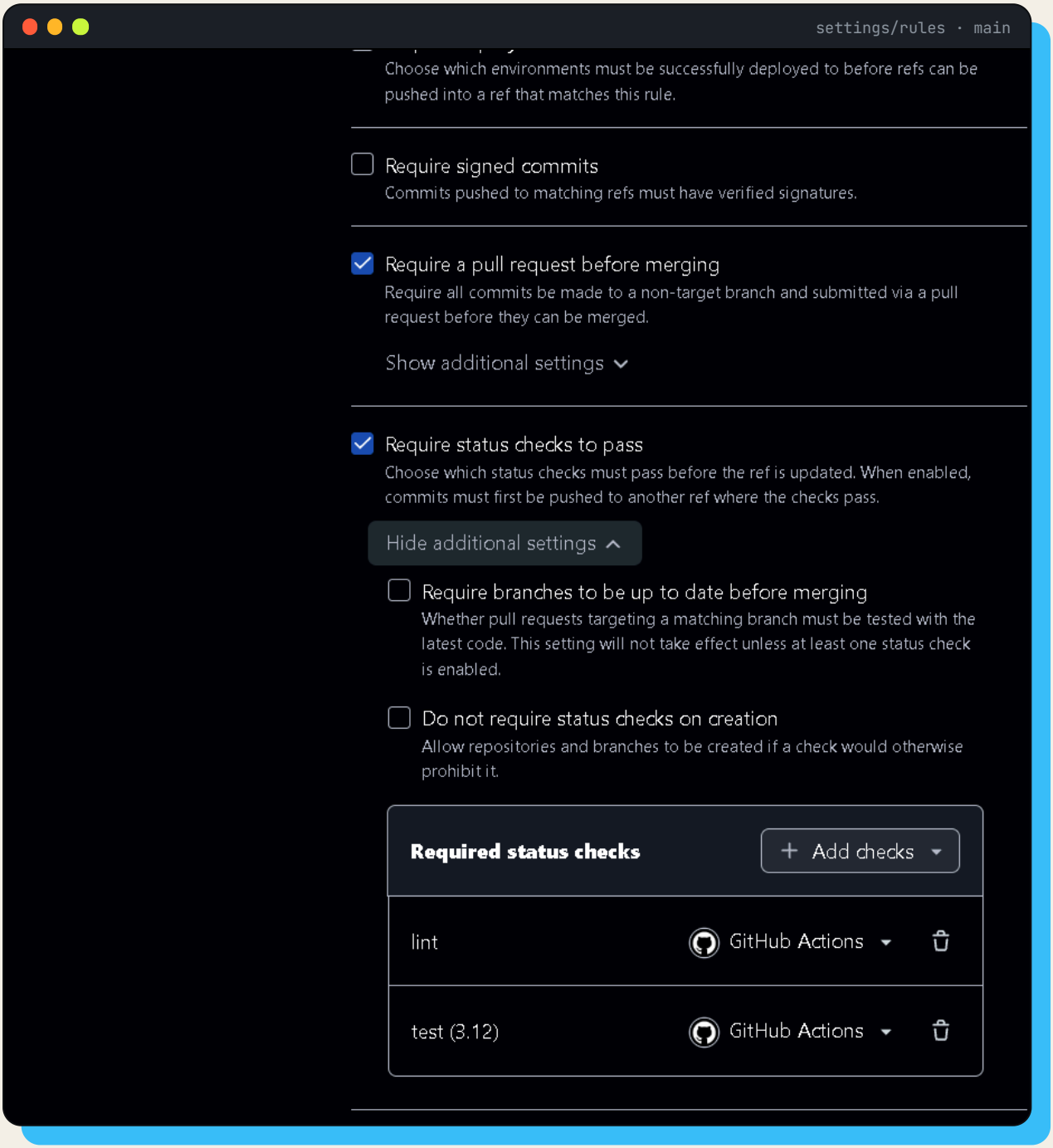




PROTECTED

TP 2-5 — `main` sous bonne garde

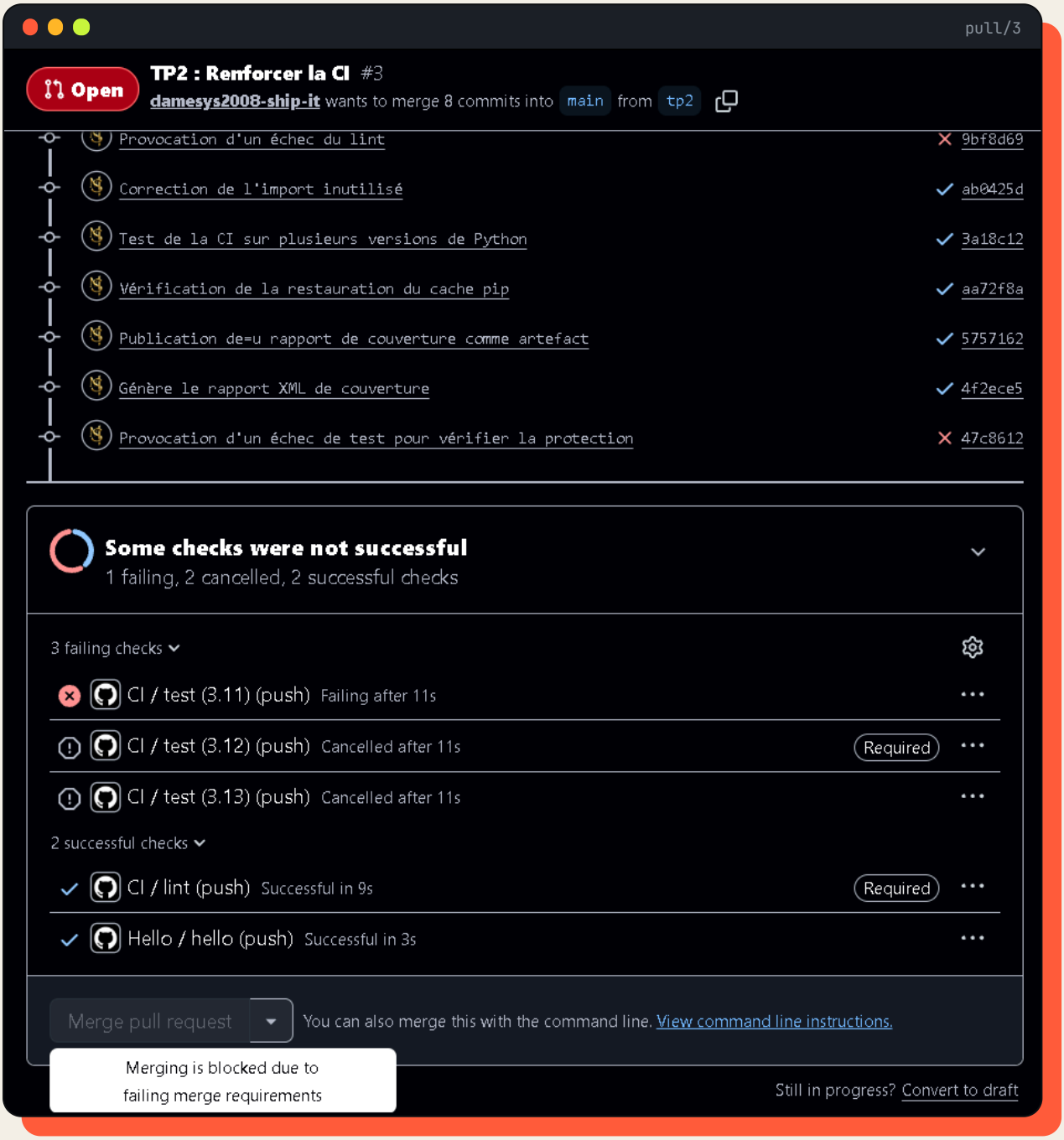
La règle de protection avec les vérifications obligatoires `lint` et `test (3.12)`.



MERGE BLOCKED

TP 2-6 Fusion refusée à l'entrée

Une pull request dont la fusion est bloquée par une vérification en échec.

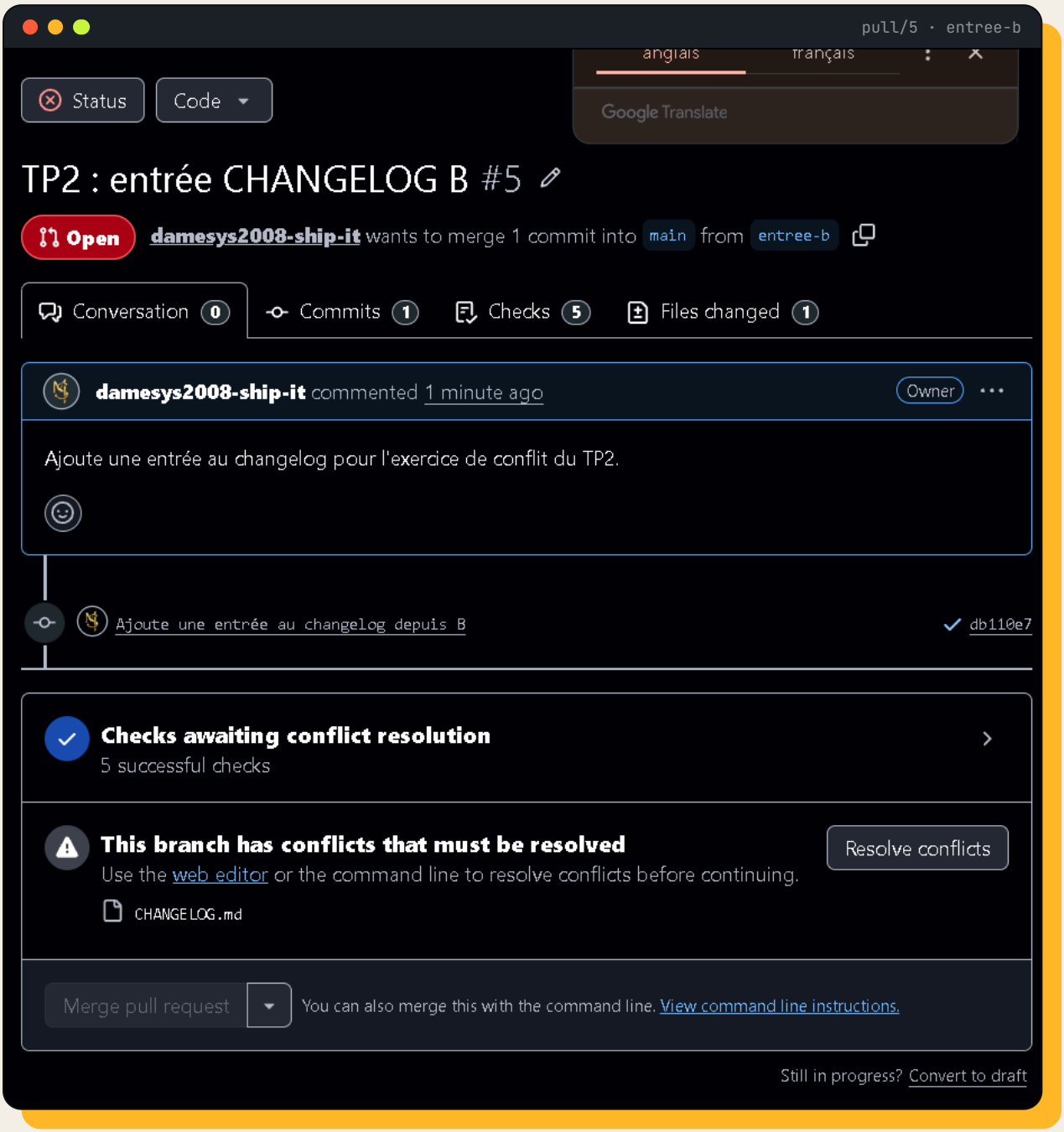


TP 2-7

CONFLICT

Deux lignes, une seule place

La PR de B avec le message de conflit sur `CHANGELOG.md`.

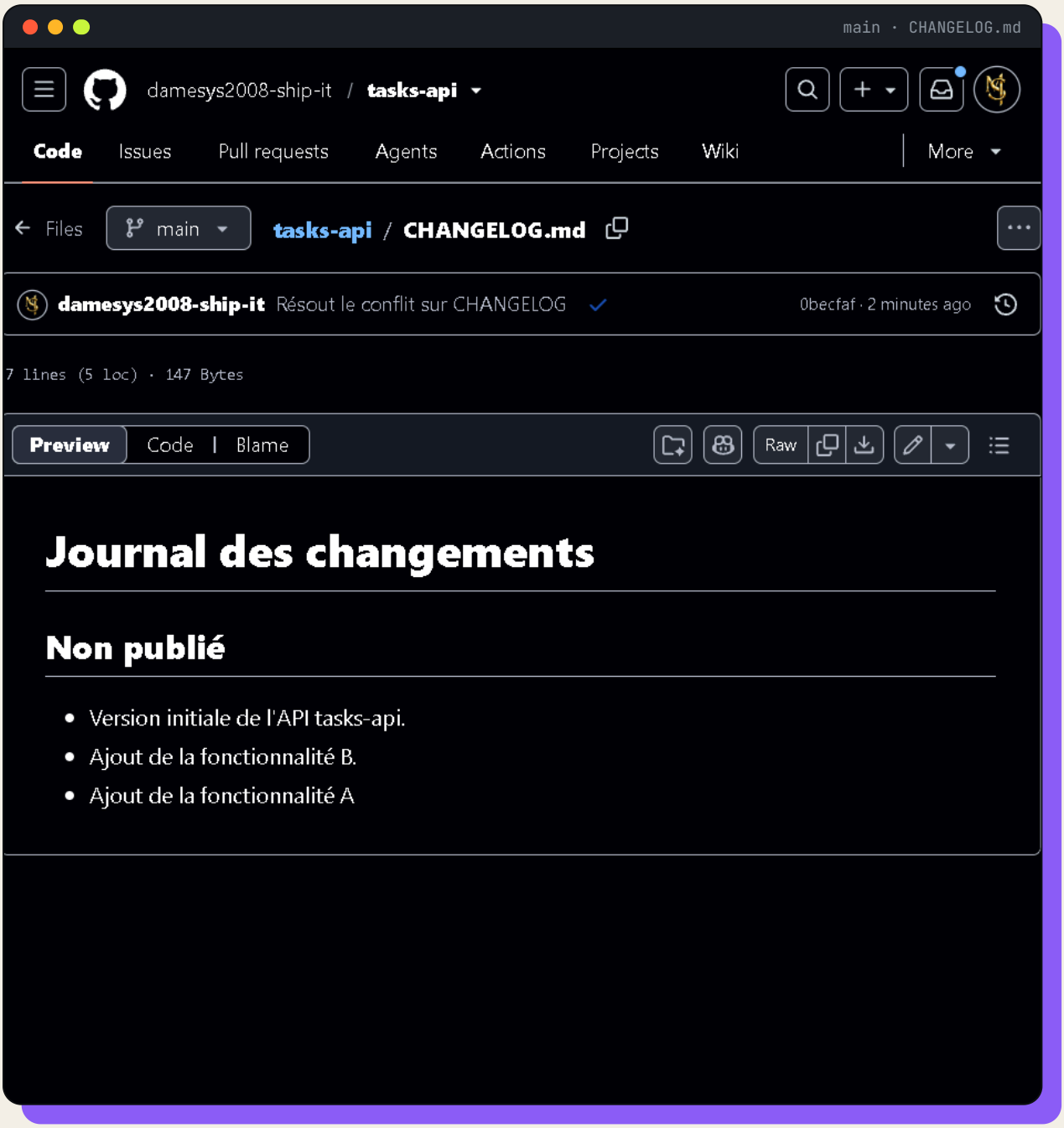


TP 2-8

MERGED

Tout le monde a sa ligne

Le fichier `CHANGELOG.md` final sur `main`, avec les deux lignes.

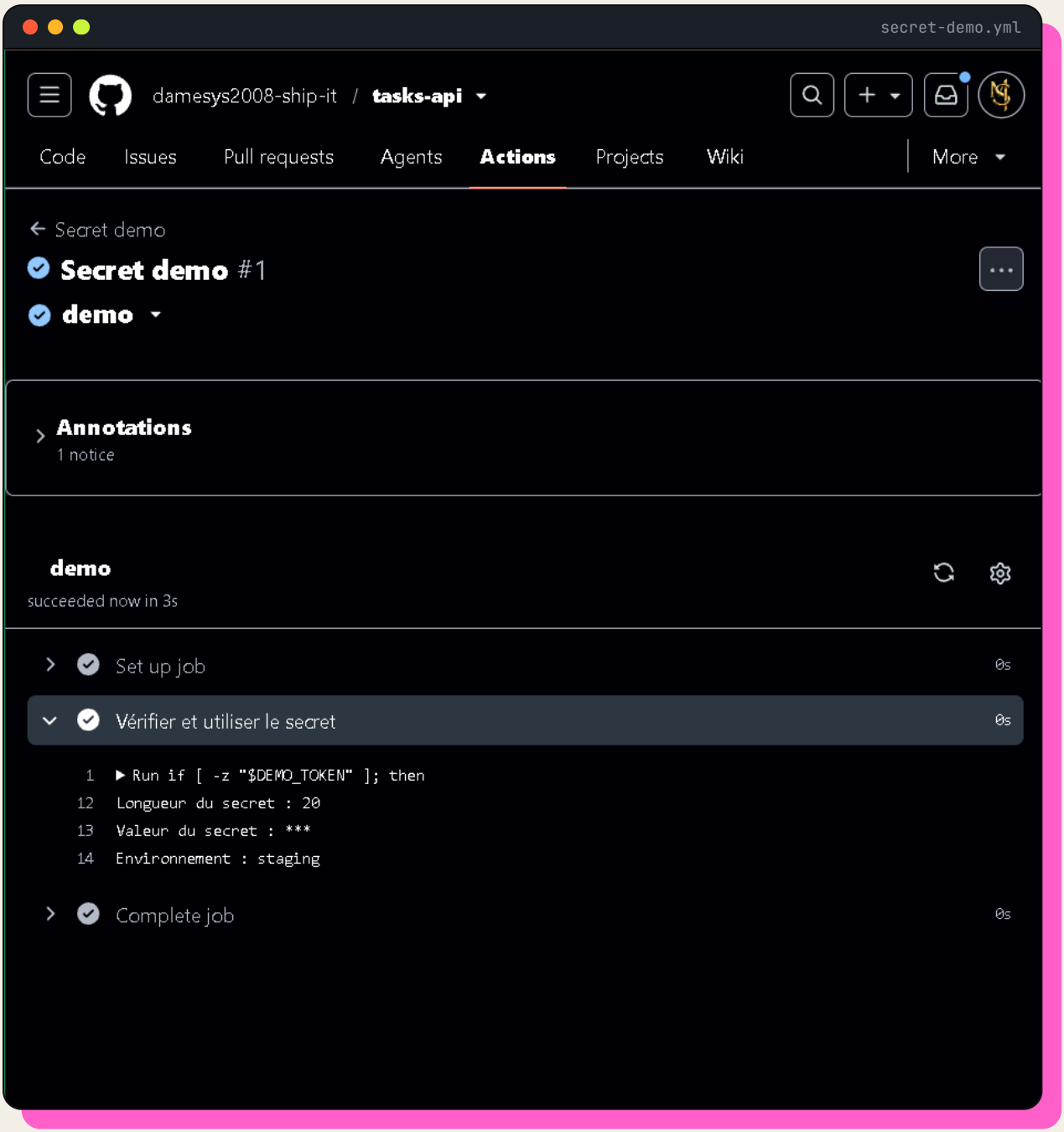


TP 2-9

✱ MASKED

Chut, c'est un secret

Les logs où la valeur du secret apparaît masquée par `***`.

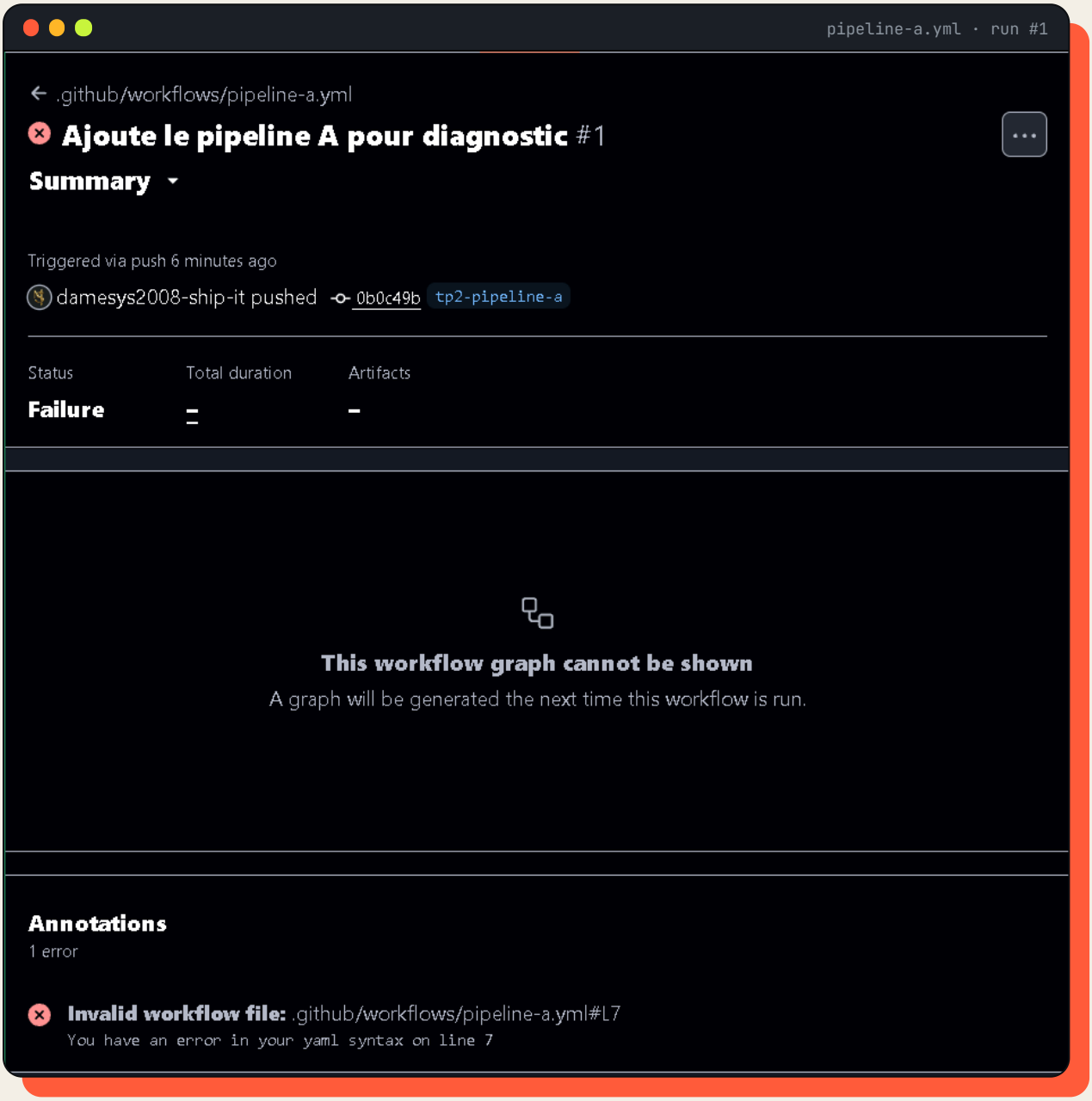


TP 2-10A

FAILED

Pipeline A : l'indentation qui fâche

Le run en échec du pipeline A : fichier de workflow invalide, erreur de syntaxe YAML ligne 7.

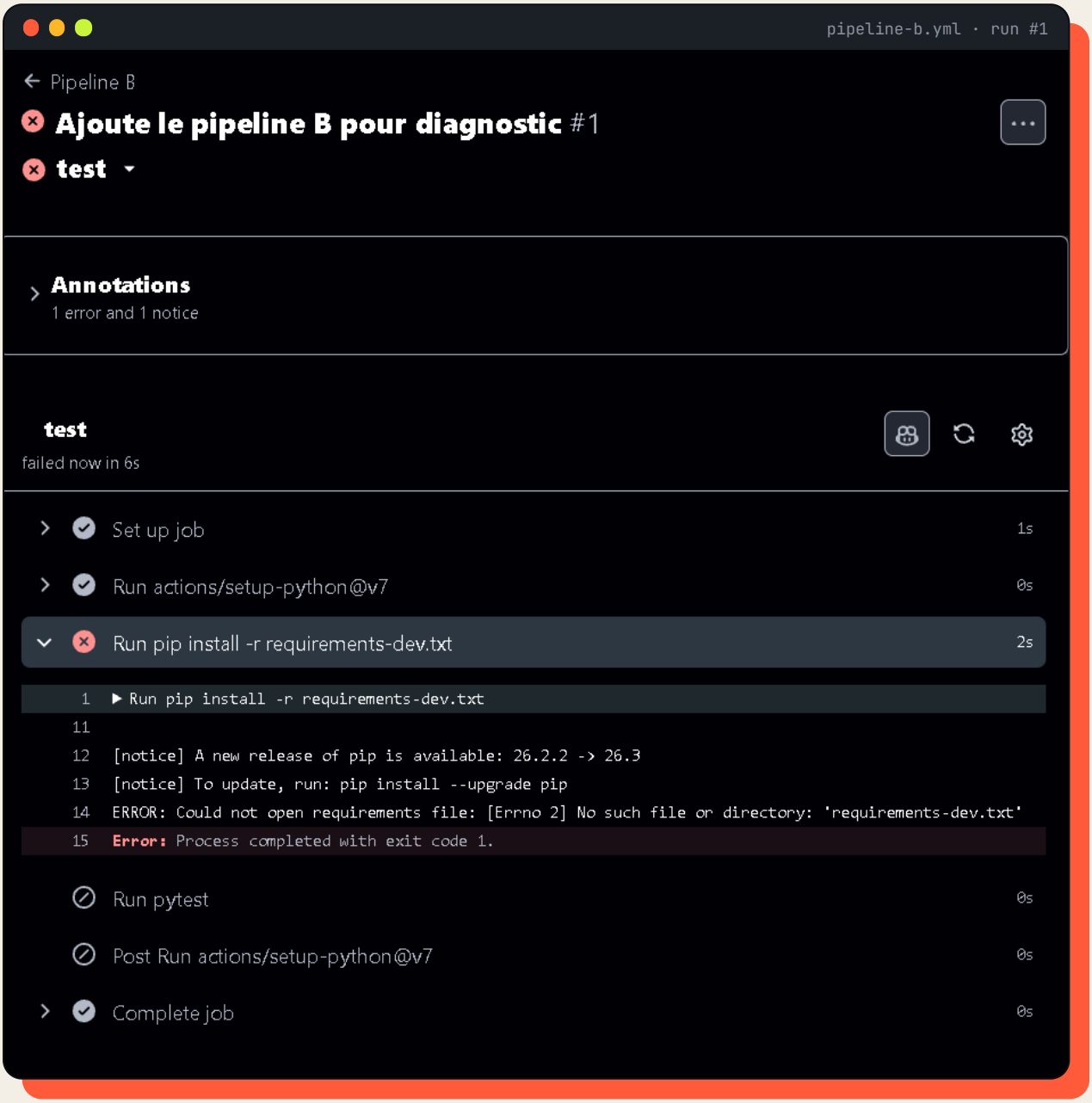


TP 2-10B

FAILED

Pipeline B : le checkout oublié

Le run en échec du pipeline B : `requirements-dev.txt` introuvable sur le runner.

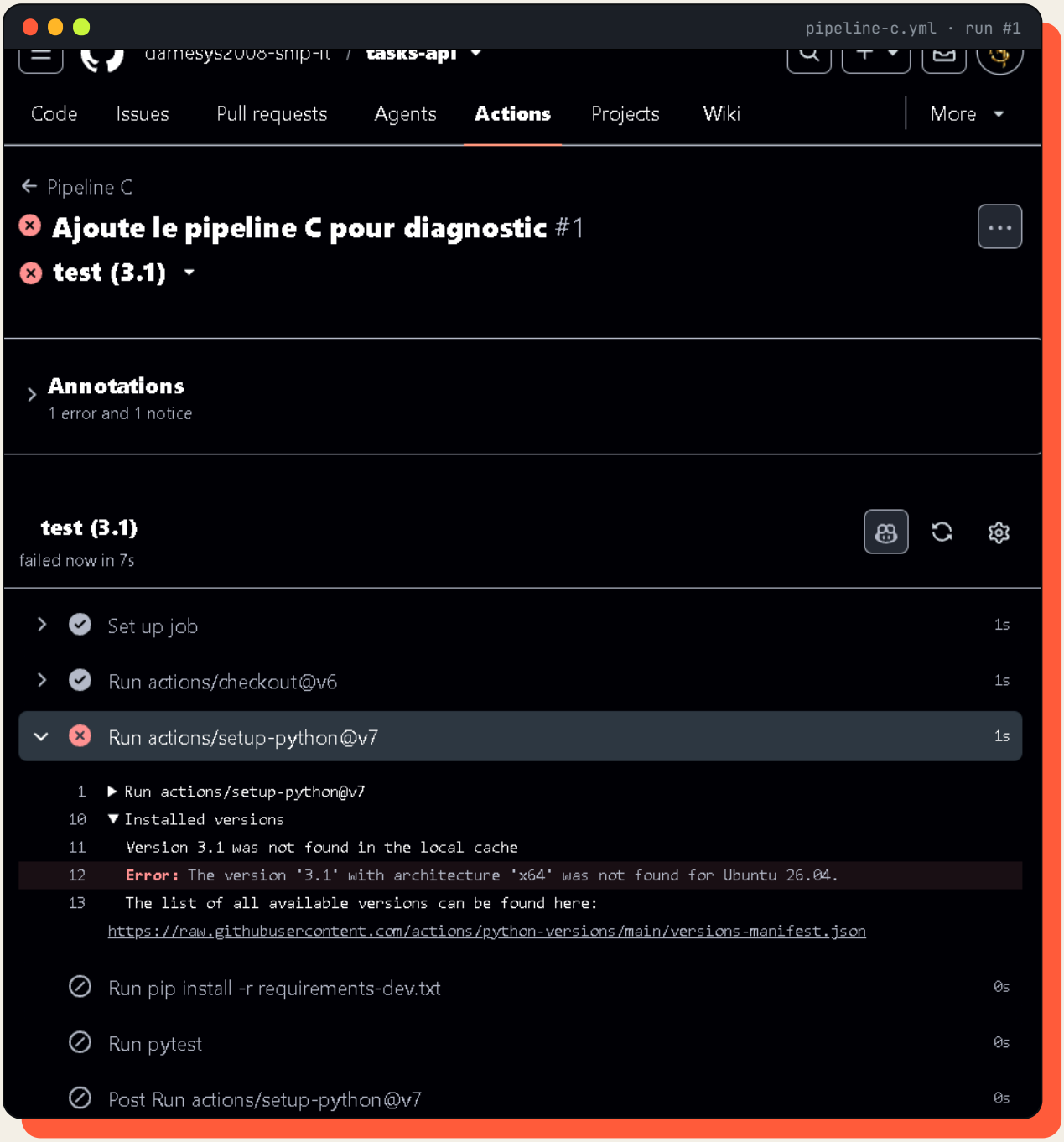


TP 2-10C

FAILED

Pipeline C : 3.10 = 3.1 ?!

Le run en échec du pipeline C : la version `'3.1'` introuvable pour setup-python.

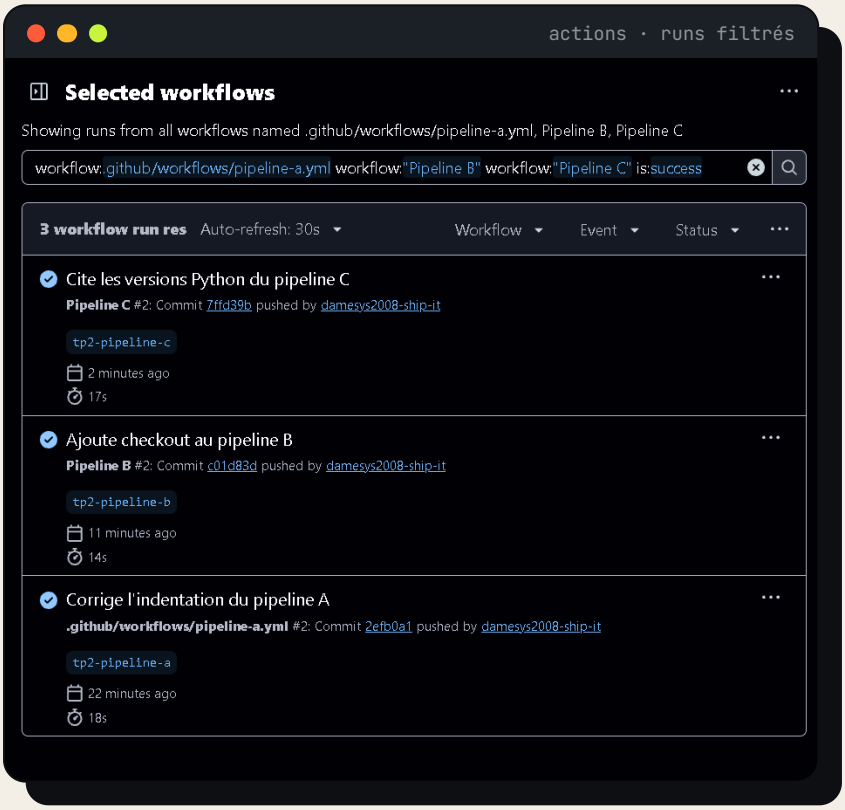


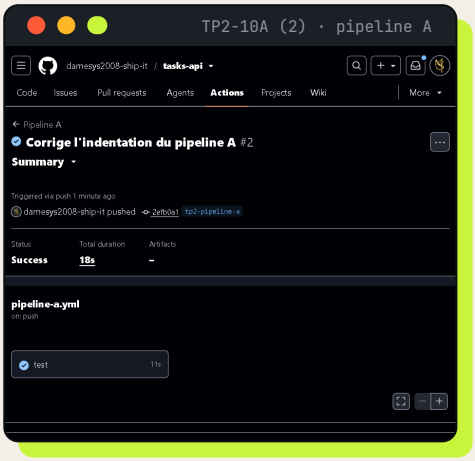
✓ ALL GREEN

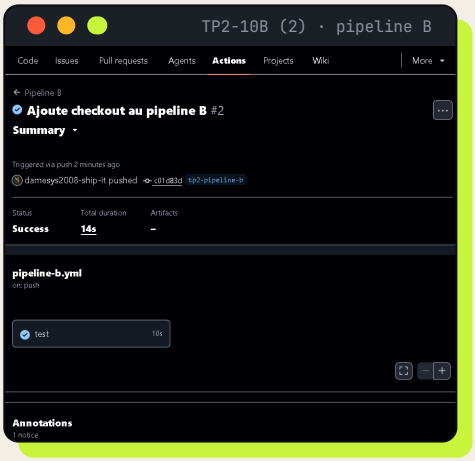
TP 2-11

Trois pipelines, trois feux verts

La liste des runs montrant les trois pipelines réparés au vert, avec à côté le run réparé de chacun (TP2-10A (2), TP2-10B (2), TP2-10C (2)).









$ cat tp2/reponses.md

Réponses
aux questions — TP2

PARTIE 1 — QUALITÉ AUTOMATIQUE

01

Quelle différence entre flake8 et Black ?

Flake8 analyse le code pour détecter des problèmes de style ou certaines erreurs, comme un import inutilisé. **Black** formate automatiquement le code selon des règles cohérentes. Dans la CI, `black --check` vérifie le formatage sans modifier les fichiers.

02

Pourquoi séparer lint et test en deux jobs ?

Cela distingue les vérifications de qualité des tests fonctionnels. Les jobs peuvent s'exécuter **en parallèle** et, si l'un échoue, ses résultats sont plus faciles à repérer sans confondre les causes.

PARTIE 2 — MATRICE, CACHE ET ARTEFACT

03

Pourquoi tester sur plusieurs versions de Python ?

Pour vérifier que l'application et ses dépendances fonctionnent sur **toutes les versions prises en charge**, et repérer une incompatibilité qui pourrait passer inaperçue avec une seule version.

04

Que gagne-t-on avec le cache ?

Le cache évite de retélécharger certains paquets déjà utilisés. Il peut ainsi **réduire le temps d'installation** et accélérer les runs suivants.

05

Un artefact, c'est quoi ?

C'est un **fichier produit par le workflow** et conservé avec le run pour pouvoir le consulter ou le télécharger ensuite. Ici, `coverage.xml` est le rapport de couverture.

PARTIE 3 — PROTECTION DE MAIN

06 Pourquoi interdire de pousser directement sur main ?

Pour faire passer les changements par une **pull request** et les **vérifications obligatoires** avant leur intégration. Cela réduit le risque d'introduire sur la branche principale un changement qui casse le projet.

PARTIE 4 — CONFLIT

07 Pourquoi Git ne peut-il pas résoudre ce conflit seul ?

Les deux branches ont modifié **la même zone du fichier** différemment. Git ne peut pas savoir si les deux changements doivent être conservés ni quel ordre est souhaité : il faut donc choisir et combiner les lignes manuellement.

08 Qu'est-ce qui a vérifié le code après la résolution ?

Les vérifications **GitHub Actions** lancées sur la pull request après le push de la résolution — notamment `lint` et les tests de la matrice — ont contrôlé le résultat avant la fusion.

PARTIE 5 — SECRETS ET VARIABLES

09 Pourquoi GitHub masque-t-il le secret ?

Pour éviter qu'une valeur enregistrée comme secret soit **révélée accidentellement dans les logs**. Ce masquage est une protection supplémentaire, pas une raison d'afficher un vrai secret.

10 Que se passe-t-il si on le commite dans le dépôt ?

La valeur peut être exposée dans le code et **rester accessible dans l'historique Git**, même si on la supprime ensuite. Il faut alors la révoquer ou la renouveler et la stocker dans les secrets GitHub, pas dans le dépôt.

11 Secret ou variable : comment choisir ?

Un **secret** sert aux données confidentielles, comme un jeton ou un mot de passe. Une **variable** convient à une configuration non sensible, comme le nom d'un environnement (`staging`).

PARTIE 6 — PIPELINES A, B ET C

PIPELINE **A**

✕ BUG & INDICE OBSERVÉ

L'indentation d'une étape YAML était décalée. GitHub a signalé une **erreur de syntaxe à la ligne 7** et le workflow ne pouvait pas s'exécuter correctement.

✓ COMMENT L'ÉVITER

Garder une indentation YAML cohérente et vérifier la syntaxe du workflow avant de le pousser.

PIPELINE **B**

✕ BUG & INDICE OBSERVÉ

L'étape `actions/checkout` manquait. Le runner ne trouvait donc pas **`requirements-dev.txt`** lors de l'installation.

✓ COMMENT L'ÉVITER

Récupérer le dépôt avec `actions/checkout` avant d'utiliser ses fichiers.

PIPELINE **C**

✕ BUG & INDICE OBSERVÉ

Les versions de la matrice n'étaient pas entre guillemets. YAML a interprété `3.10` comme le **nombre 3.1**, que setup-python ne pouvait pas installer.

✓ COMMENT L'ÉVITER

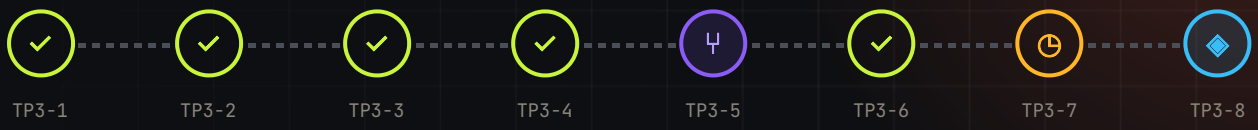
Écrire les versions comme chaînes, par exemple `"3.10"`, et vérifier leur valeur dans la matrice.

TP2 — 3/3 pipelines réparés · `exit 0`

03

```
$ docker build -t tasks-api:local .
```

Docker & *livraison* continue



OBJECTIF

Mettre l'application dans une image Docker, publier automatiquement cette image à chaque changement, créer une version (release) par un tag Git, et déployer avec une approbation avant la production.

CAPTURES
11

DÉPÔT
damesys2008-ship-it/tasks-api

CI passing

✓ IMAGE BUILT

TP 3-1

Mise en boîte

Le `docker build` réussi, puis `curl` sur `/health` qui répond `{"status":"ok"}`.

PowerShell · docker build && docker run

```
(.venv) PS C:\Users\dames\Downloads\tasks-api> docker build -t tasks-api:local .
[+] Building 25.3s (12/12) FINISHED                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                 0.1s
 => => transferring dockerfile: 1.31kB                                                               0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                  1.8s
 => [auth] library/python:pull token for registry-1.docker.io                                        0.0s
 => [internal] load .dockerignore                                                                    0.0s
 => => transferring context: 196B                                                                    0.0s
 => [1/6] FROM docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded85abb22c2b  11.0s
 => => resolve docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded85abb22c2b  0.0s
 => => sha256:acca7542dfc736e9ce20cbefc58b2d09822276d15d4a0c531531a6f1b057928e 251B / 251B           0.1s
 => => sha256:f574edac59d9f092df596f54e5b52656c803fd3ae30825b8ee5a189ed05f63d3 12.12MB / 12.12MB     3.4s
 => => sha256:3eb77115150676568f88e2957181b8f7e3014e797f1ce8cd640086dfbed45e35 1.29MB / 1.29MB       1.2s
 => => sha256:6fc242d221a673263eabd14a9ad9584d3177c96e5cb504c891497338e2f912f5 29.86MB / 29.86MB     7.3s
 => => extracting sha256:6fc242d221a673263eabd14a9ad9584d3177c96e5cb504c891497338e2f912f5            2.0s
 => => extracting sha256:3eb77115150676568f88e2957181b8f7e3014e797f1ce8cd640086dfbed45e35            0.3s
 => => extracting sha256:f574edac59d9f092df596f54e5b52656c803fd3ae30825b8ee5a189ed05f63d3            1.1s
 => => extracting sha256:acca7542dfc736e9ce20cbefc58b2d09822276d15d4a0c531531a6f1b057928e            0.0s
 => [internal] load build context                                                                    0.1s
 => => transferring context: 6.24kB                                                                  0.0s
 => [2/6] WORKDIR /app                                                                               0.5s
 => [3/6] COPY requirements.txt .                                                                    0.1s
 => [4/6] RUN pip install --no-cache-dir -r requirements.txt                                         5.3s
 => [5/6] COPY app/ ./app/                                                                           0.1s
 => [6/6] RUN useradd --create-home appuser                                                          0.4s
 => exporting to image                                                                               5.8s
 => => exporting layers                                                                              1.3s
 => => exporting manifest sha256:5acdb03d329e1789e71c0d4376745cfa86f78f185267d380f7ea317f664c85e3    0.0s
 => => exporting config sha256:8ee54867bb0aec562d3def358dc2ab57b2f8a779caed7fe121bd92a7bb9f4f84      0.0s
 => => exporting attestation manifest sha256:cd1a07821ed6907c2963d52496aa3339c9edd8842c3878ed08e3e8  0.0s
 => => exporting manifest list sha256:1bb40ec03a08fb0ec453a39d7ebd1f91e3ba4a4b1faa56d933e7766fe4563  0.0s
 => => naming to docker.io/library/tasks-api:local                                                   0.0s
 => => unpacking to docker.io/library/tasks-api:local                                                4.3s
(.venv) PS C:\Users\dames\Downloads\tasks-api> docker run --rm -p 8000:8000 tasks-api:local
[2026-11-04 14:15:59 +0000] [1] [INFO] Starting gunicorn 26.2.0
[2026-11-04 14:15:59 +0000] [1] [INFO] Listening at: http://0.0.0.0:8000 (1)
[2026-11-04 14:15:59 +0000] [1] [INFO] Using worker: sync
[2026-11-04 14:15:59 +0000] [7] [INFO] Booting worker with pid: 7
[2026-11-04 14:15:59 +0000] [1] [INFO] Control socket listening at /home/appuser/.gunicorn/gunicorn.ctl
```

PowerShell · curl.exe

```
(.venv) PS C:\Users\dames\Downloads\tasks-api> curl.exe http://localhost:8000/health
{"status":"ok"}
(.venv) PS C:\Users\dames\Downloads\tasks-api> curl.exe http://localhost:8000/version
{"version":"dev"}
(.venv) PS C:\Users\dames\Downloads\tasks-api>
```

COMPOSE UP

TP 3-2 — Un fichier pour les lancer tous

`docker compose up` en cours, et le `curl` sur `/version` qui répond `local`.

PowerShell · docker compose up --build

```
(.venv) PS C:\Users\dames\Downloads\tasks-api> docker compose up --build
[+] Building 2.7s (14/14) FINISHED
 => [internal] load local bake definitions
 => => reading from stdin 521B
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 1.31kB
 => [internal] load metadata for docker.io/library/python:3.12-slim
 => [auth] library/python:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 196B
 => [1/6] FROM docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded85abb22c2b8b262da527dda1d031ab30
 => => resolve docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded85abb22c2b8b262da527dda1d031ab30
 => [internal] load build context
 => => transferring context: 1.67kB
 => CACHED [2/6] WORKDIR /app
 => CACHED [3/6] COPY requirements.txt .
 => CACHED [4/6] RUN pip install --no-cache-dir -r requirements.txt
 => CACHED [5/6] COPY app/ ./app/
 => CACHED [6/6] RUN useradd --create-home appuser
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:e3254c556774717287cc757ae77dbccb459c0443bccffa2a0a2cc993371fc891
 => => exporting config sha256:bd220e6de3e9908694da0056e1e19432b0882296faf591d2b5b3cf9762210557
 => => exporting attestation manifest sha256:16a4370d23fd35f3e035e375b5d71fb05dd2f47d6b8c8f15ecd287240b7d0900
 => => exporting manifest list sha256:400750e500837125adc2d389c11cd801d510545666a485a9dbbd897f00cd7bb3
 => => naming to docker.io/library/tasks-api-api:latest
 => => unpacking to docker.io/library/tasks-api-api:latest
 => resolving provenance for metadata file
[+] up 3/3
 ✔ Image tasks-api-api        Built
 ✔ Network tasks-api_default  Created
 ✔ Container tasks-api-api-1  Created
Attaching to api-1
api-1  | [2026-11-04 14:27:45 +0000] [1] [INFO] Starting gunicorn 26.2.0
api-1  | [2026-11-04 14:27:45 +0000] [1] [INFO] Listening at: http://0.0.0.0:8000 (1)
api-1  | [2026-11-04 14:27:45 +0000] [1] [INFO] Using worker: sync
api-1  | [2026-11-04 14:27:45 +0000] [7] [INFO] Booting worker with pid: 7
api-1  | [2026-11-04 14:27:45 +0000] [1] [INFO] Control socket listening at /home/appuser/.gunicorn/gunicorn.ctl

v View in Docker Desktop   o View Config   w Enable Watch   d Detach
```

PowerShell · curl.exe

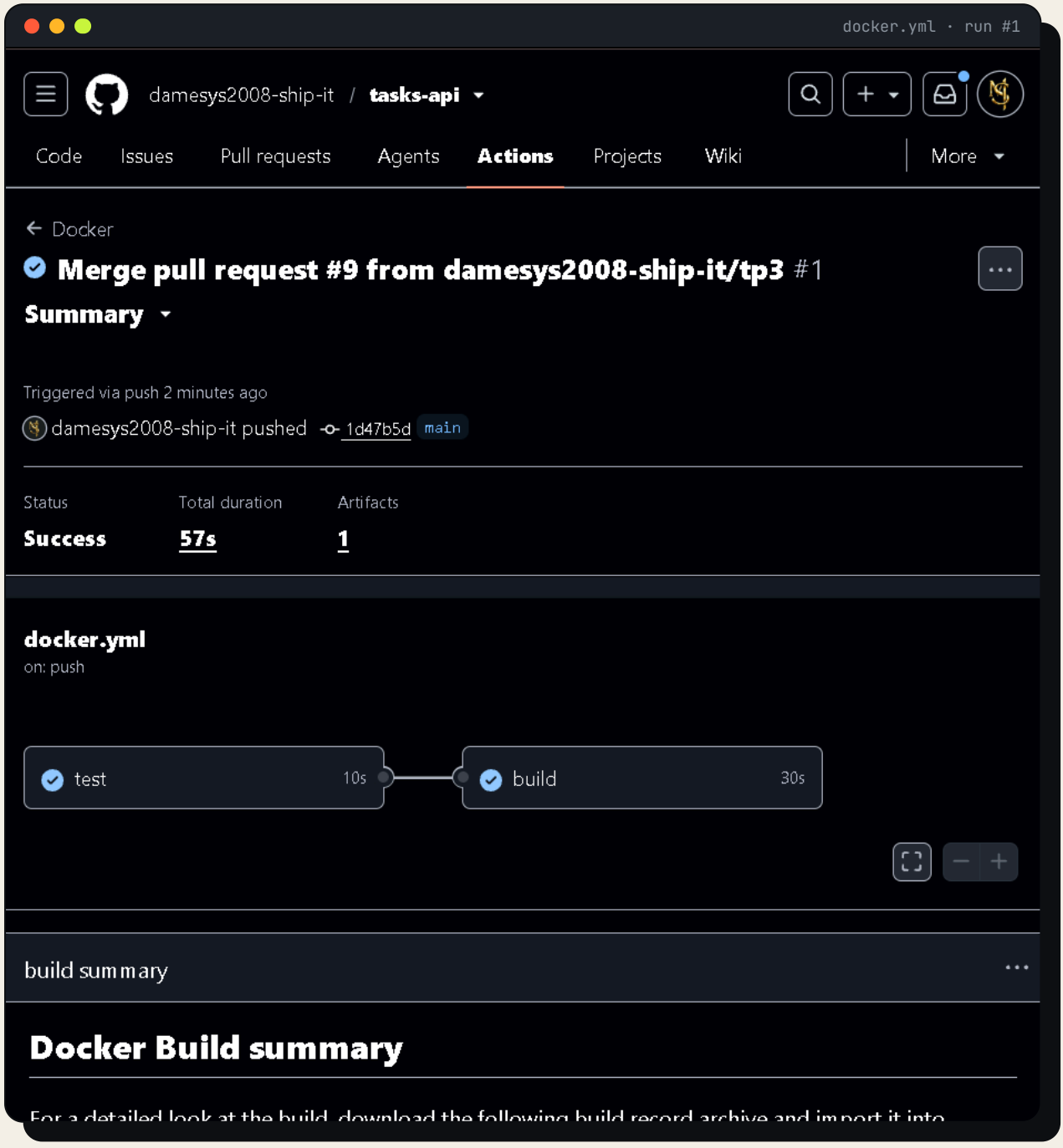
```
(.venv) PS C:\Users\dames\Downloads\tasks-api> curl.exe http://localhost:8000/version
{"version":"local"}
(.venv) PS C:\Users\dames\Downloads\tasks-api>
```

TP 3-3

PUBLISHED

test, puis build

Le run `docker.yml` avec `test` puis `build` au vert.

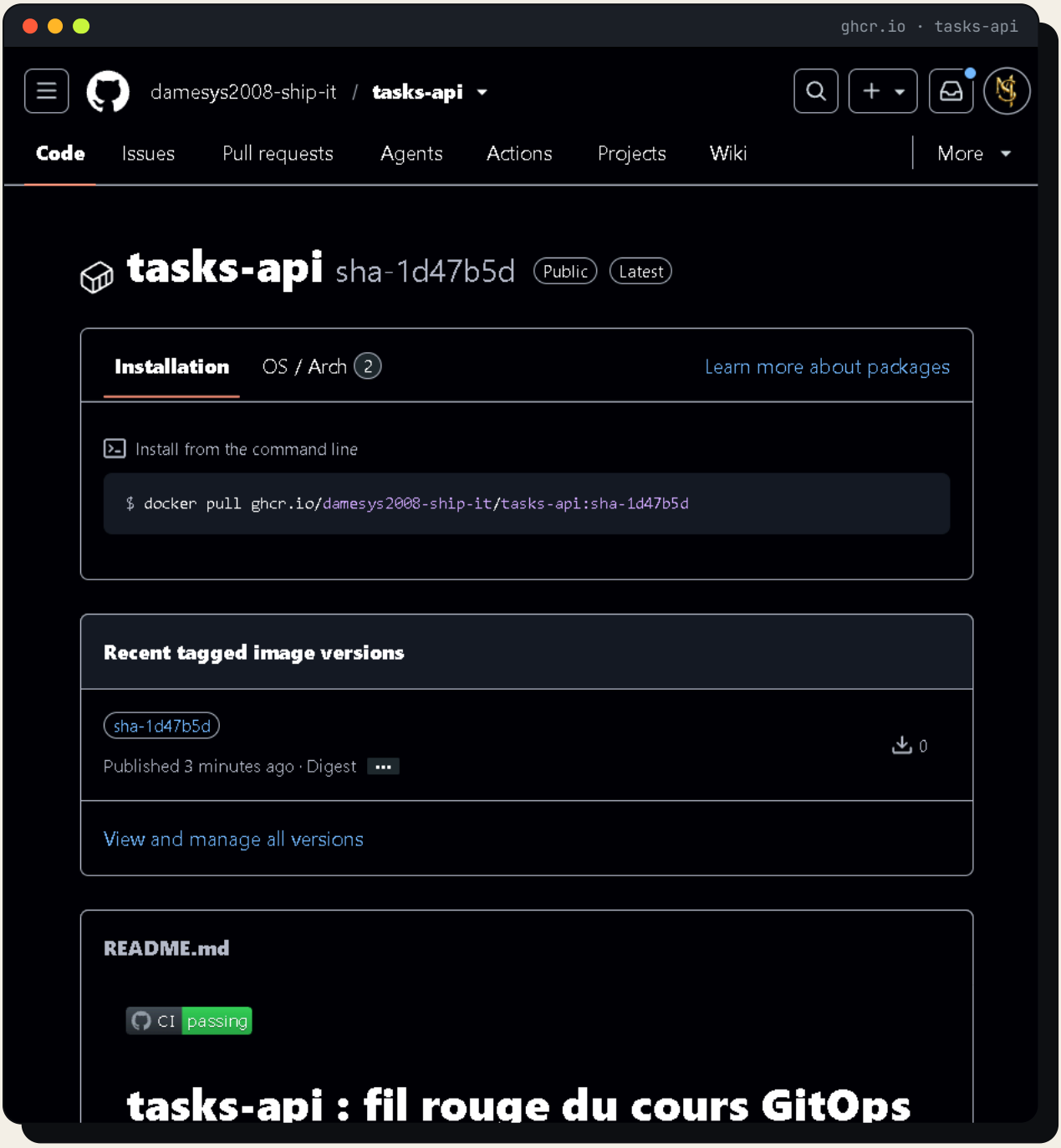


TP 3-4

PUBLISHED

Livré sur GHCR

La page du package `tasks-api` avec son tag `sha-1d47b5d`.

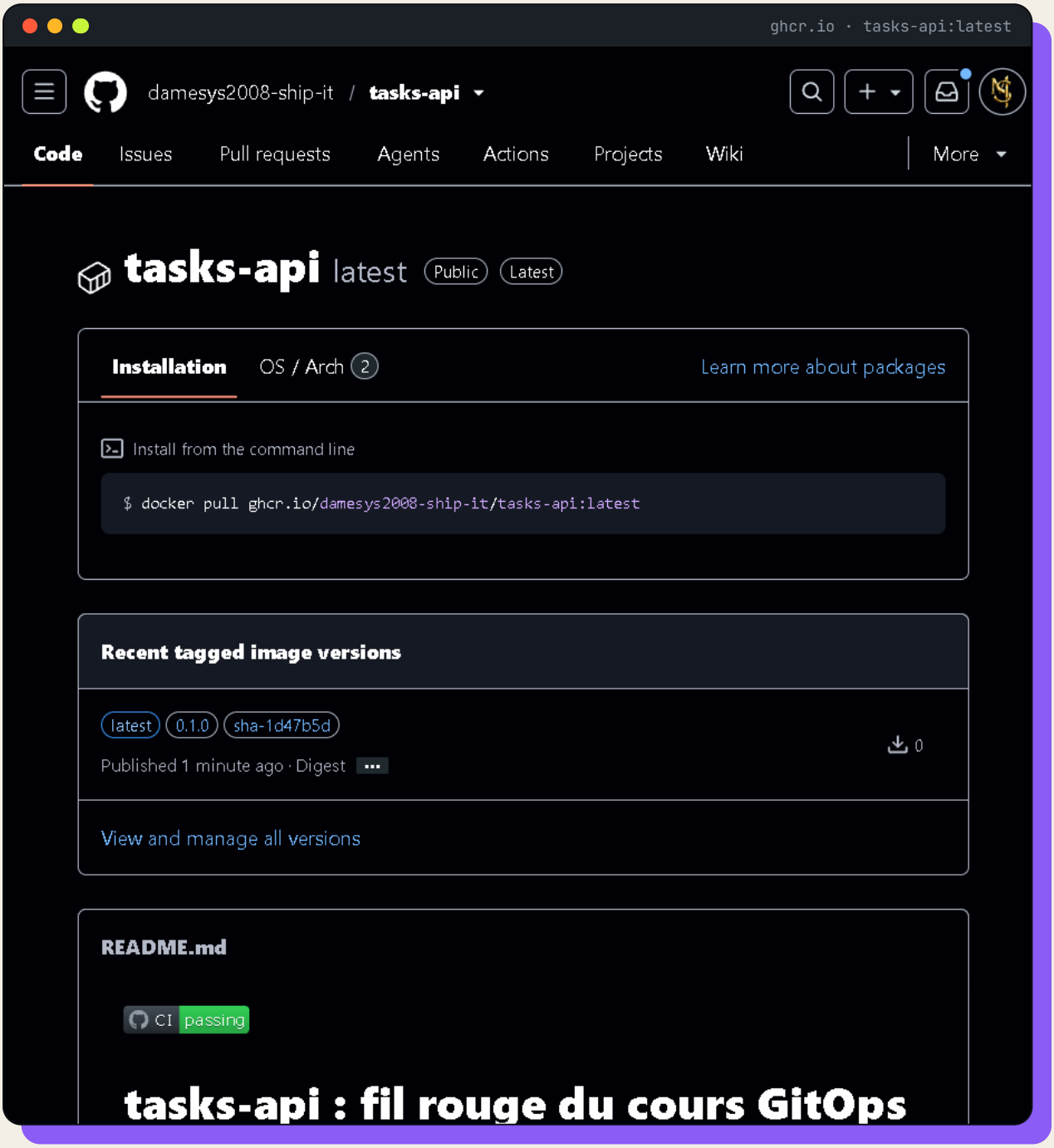


TP 3-5

RELEASE

v0.1.0, officiellement

La liste des tags du package : `0.1.0` et `sha-1d47b5d` (et `latest`).

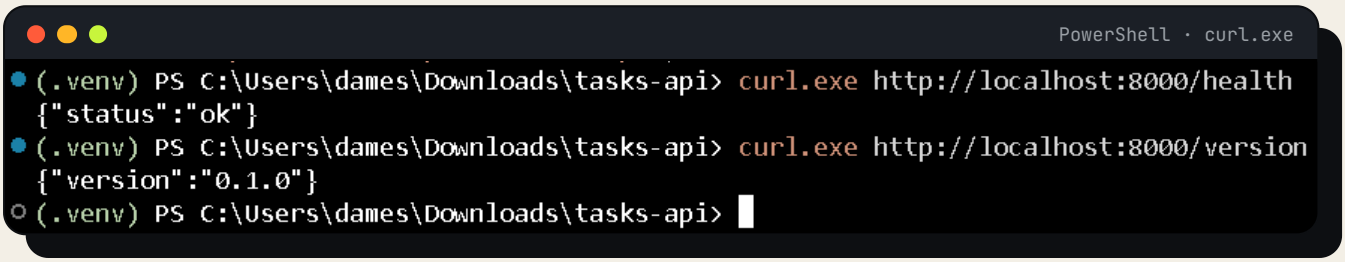


TP 3-6

✓ 0.1.0 LIVE

Comme un vrai utilisateur

L'image publiée sur GHCR : `curl /version` répond `0.1.0`.

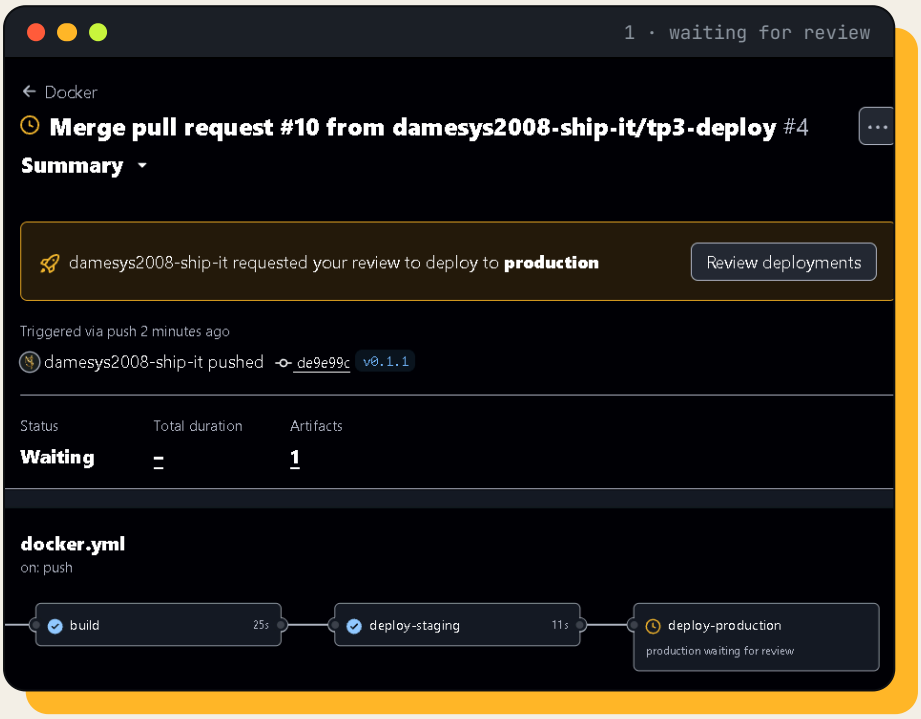


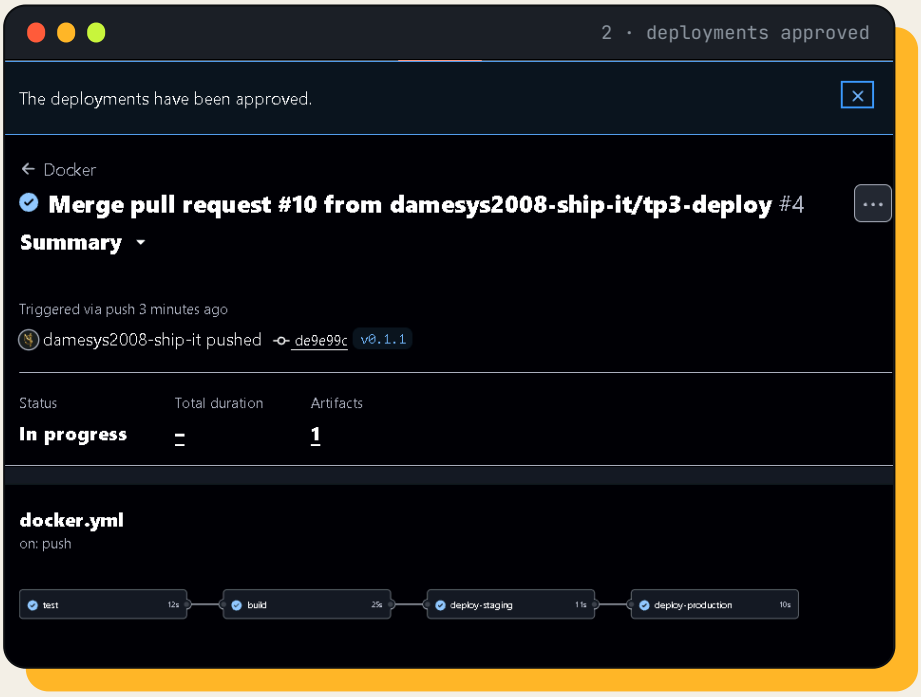
→ `{"version":"0.1.0"}`

APPROVAL

TP 3-7 La prod attend ton feu vert

Le run en attente d'approbation (« Review deployments »), puis le run approuvé avec `deploy-production` au vert.



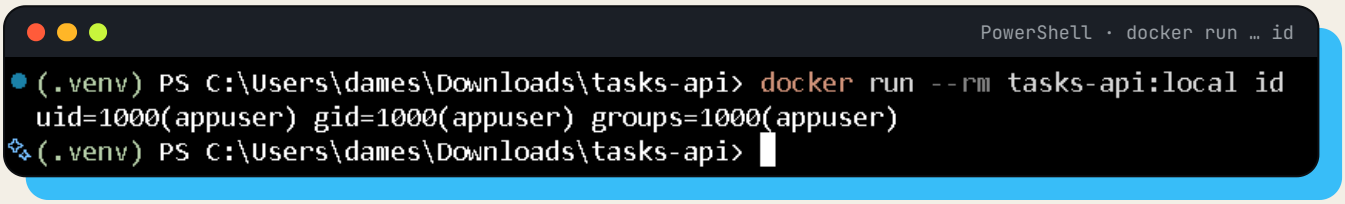


NON-ROOT

TP 3-8

Pas de root à bord

La sortie de la commande `id` dans le conteneur : un uid différent de 0.



→ uid=1000(appuser)

$ cat tp3/reponses.md

Réponses
aux questions — TP3

PARTIE 1 — DOCKERFILE

01 Quelle différence entre une image et un conteneur ?

Une **image** est un modèle construit à partir du Dockerfile, contenant l'application et ses dépendances. Un **conteneur** est une instance en cours d'exécution créée depuis cette image. Plusieurs conteneurs peuvent provenir de la même image.

02 À quoi sert le `.dockerignore` ?

Il exclut du contexte de construction les fichiers inutiles ou sensibles, comme `.git`, `.venv` et les caches. Cela **réduit le transfert**, accélère le build et évite d'intégrer des fichiers locaux dans l'image.

03 Pourquoi copier `requirements.txt` avant le code ? Quelles étapes sont en cache ?

Docker **met en cache les couches**. Comme les dépendances changent moins souvent que le code, leur copie et leur installation peuvent rester en cache quand seul un fichier de l'application change. Docker reconstruit alors les couches à partir de la copie du code.

PARTIE 2 — DOCKER COMPOSE

04 Que gagne-t-on à décrire le lancement dans un fichier ?

La configuration des ports, des variables d'environnement et de la construction est conservée **au même endroit** et peut être partagée et rejouée facilement. Une commande comme `docker compose up` évite de retaper une longue commande `docker run` et réduit les différences entre lancements.

PARTIE 3 — PUBLICATION SUR GHCR

05 Pourquoi `build` attend-il `test` ?

La dépendance `needs: test` **empêche de publier une image si les tests échouent**. Cela évite de livrer une version déjà connue comme invalide.

06 Pourquoi éviter le tag `latest` en production ?

`latest` ne précise pas quelle version exacte il désigne et peut pointer vers un contenu différent au fil du temps. Un **tag versionné** permet d'identifier, de reproduire et de restaurer précisément une release.

$ cat tp3/reponses.md — suite

PARTIE 4 — IMAGE VERSIONNÉE

07 Quel est l'avantage d'une image versionnée par rapport au code de ma machine ?

Une image versionnée est un **artefact construit et identifié de façon reproductible**. Elle peut être téléchargée et exécutée dans différents environnements sans dépendre de l'état du poste où le code a été développé.

PARTIE 5 — ENVIRONNEMENTS ET APPROBATION

08 À quoi sert un test de fumée ?

C'est une **vérification rapide des fonctions essentielles** après le déploiement. Ici, les appels à `/health` et `/version` confirment que le conteneur démarre et que l'API répond.

09 Pourquoi demander une approbation humaine avant la production ?

Elle ajoute **un contrôle avant une mise en production** : une personne peut vérifier que les tests et le staging sont satisfaisants, et que le moment est approprié.

PARTIE 6 — SÉCURITÉ

10 Quelles bonnes pratiques retenir de l'incident Trivy ?

Épingler les actions à une version fiable, **limiter les permissions** du workflow au strict nécessaire et ne pas exposer de secrets inutiles. Un composant tiers utilisé dans la CI doit être traité comme une dépendance de la chaîne logicielle.

11 Pourquoi ne pas exécuter une application en root dans un conteneur ?

Si l'application est compromise, un **utilisateur non-root limite les actions possibles** dans le conteneur. Cela réduit l'impact potentiel, même si ce n'est pas à lui seul une frontière de sécurité complète.

TP3 — image livrée, prod approuvée · `exit 0`

$ git tag v0.2.0 && git push origin v0.2.0

GitOps *avec* Argo CD



OBJECTIF

Déployer l'application avec le GitOps : on ne touche plus jamais au cluster à la main, on modifie un dépôt Git et Argo CD applique le changement. À la fin, un simple `git tag` déploie une nouvelle version dans le cluster.

CAPTURES
10

DÉPÔT
damesys2008-ship-it/tasks-api

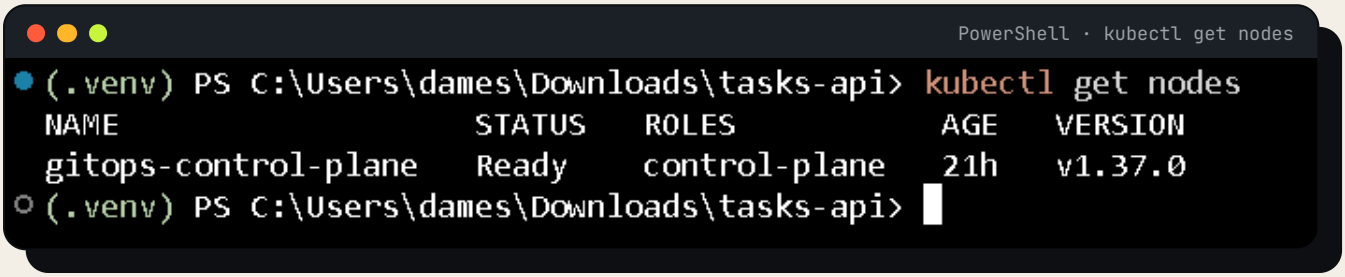
CI passing

TP 4-1

NODE READY

Un cluster dans Docker

`kubectl get nodes` : le nœud `gitops-control-plane` est `Ready`.

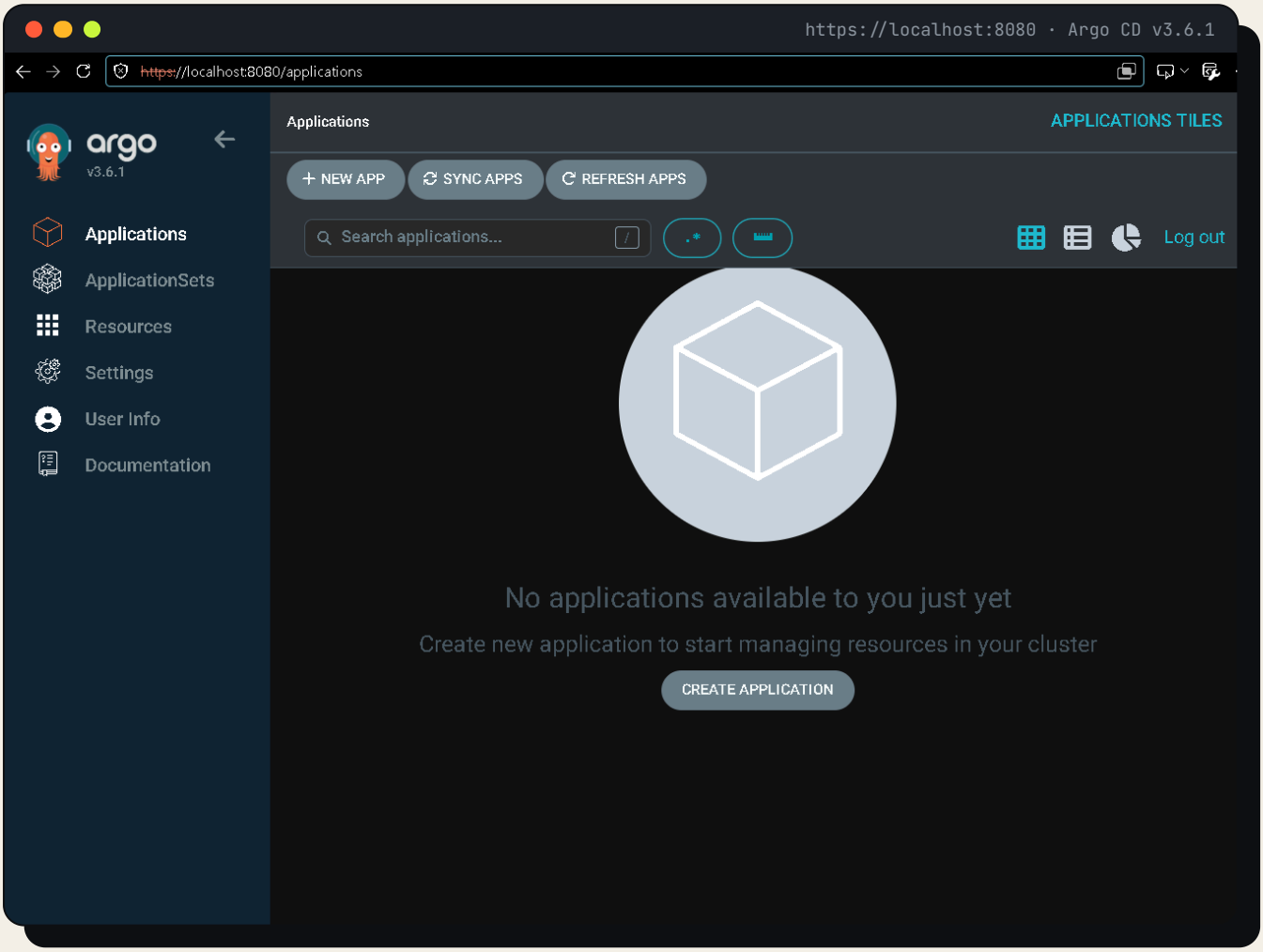


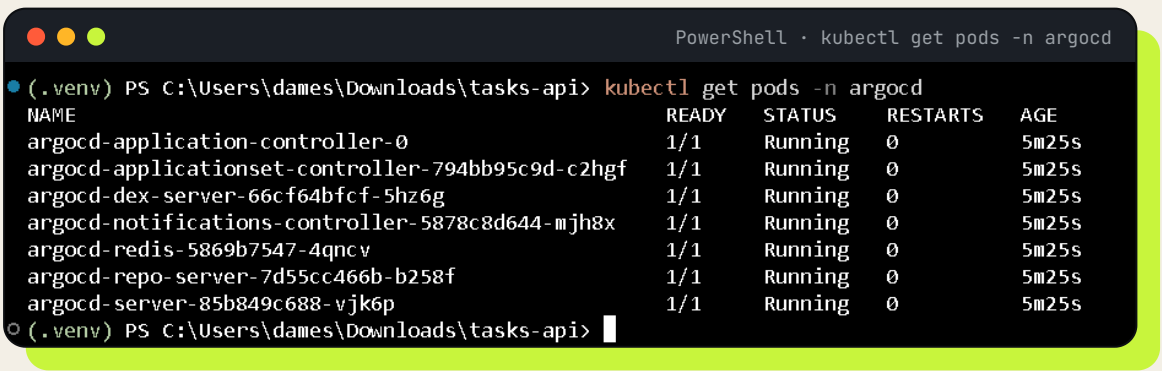
→ **gitops-control-plane Ready**

ARGO CD UP

TP 4-2 — Argo CD prend ses quartiers

`kubectl get pods -n argocd` avec tous les pods `Running`, et l'interface d'Argo CD sur `localhost:8080`.

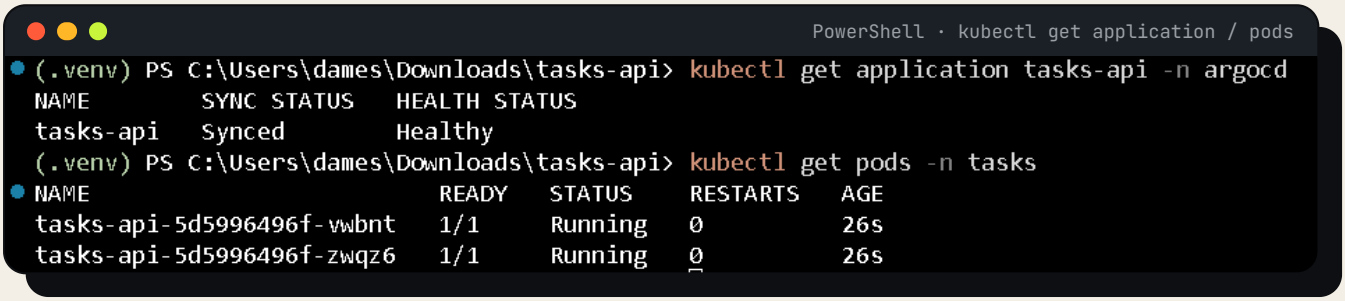




✓ SYNCED · HEALTHY

TP 4-3 Synced, Healthy, deux pods

L'application `tasks-api` à l'état `Synced` et `Healthy`, avec ses 2 pods `Running` dans le namespace `tasks`.



→ Synced · Healthy · 2/2

TP 4-4

✓ 0.1.0 LIVE

L'API répond depuis le cluster

La réponse de `/version` obtenue dans le cluster, via le tunnel `port-forward` sur le port 8081.

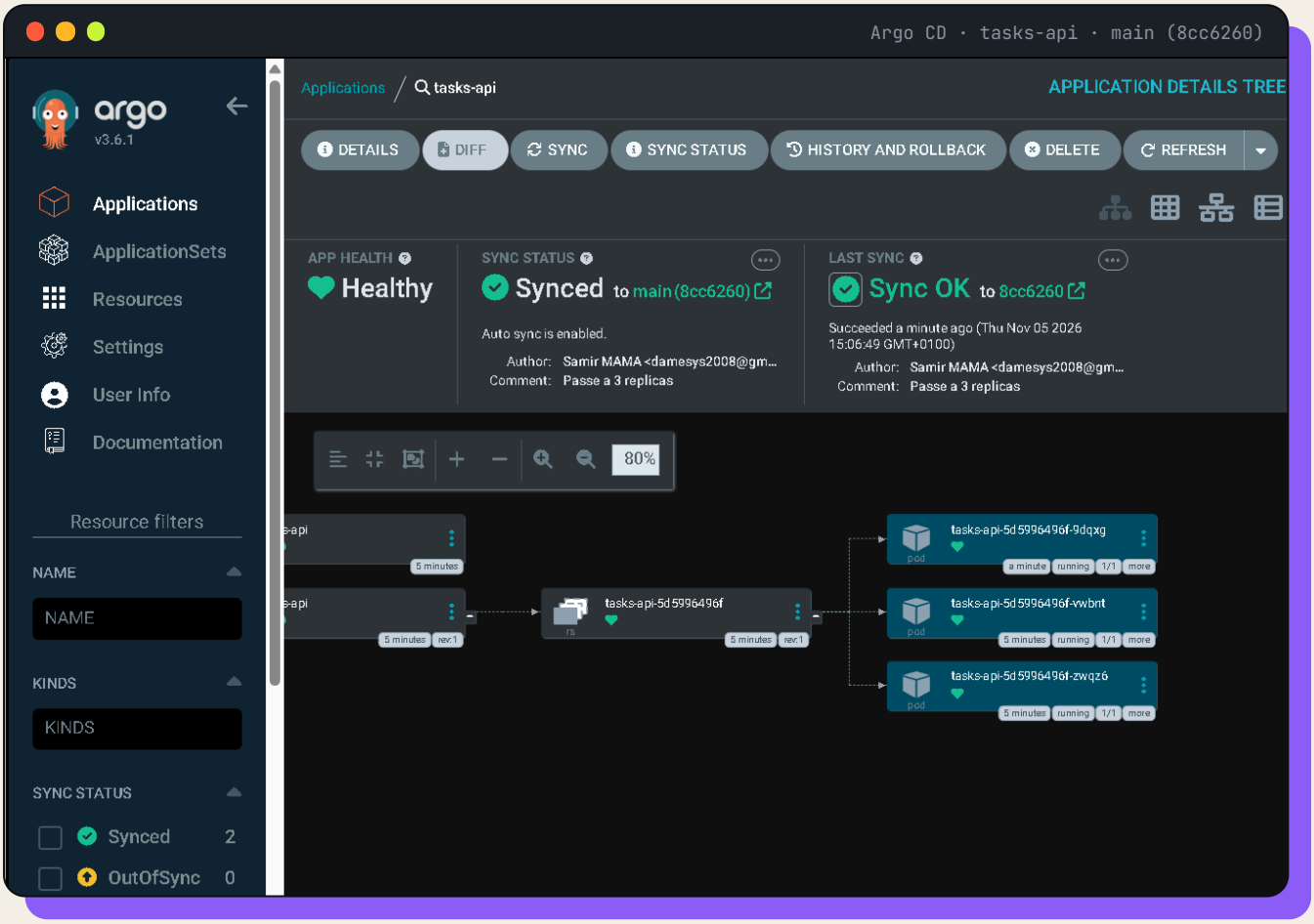


→ `{"version":"0.1.0"}`

REPLICAS: 3

TP 4-5 — Un commit, un pod de plus

Le commit « Passe a 3 replicas » synchronisé par Argo CD, avec les 3 pods dans l'arbre de l'application.



SELF-HEAL

TP 4-6 La dérive ne dure pas

Observation de la dérive manuelle : `kubectl scale … --replicas=5` face à `selfHeal: true`.

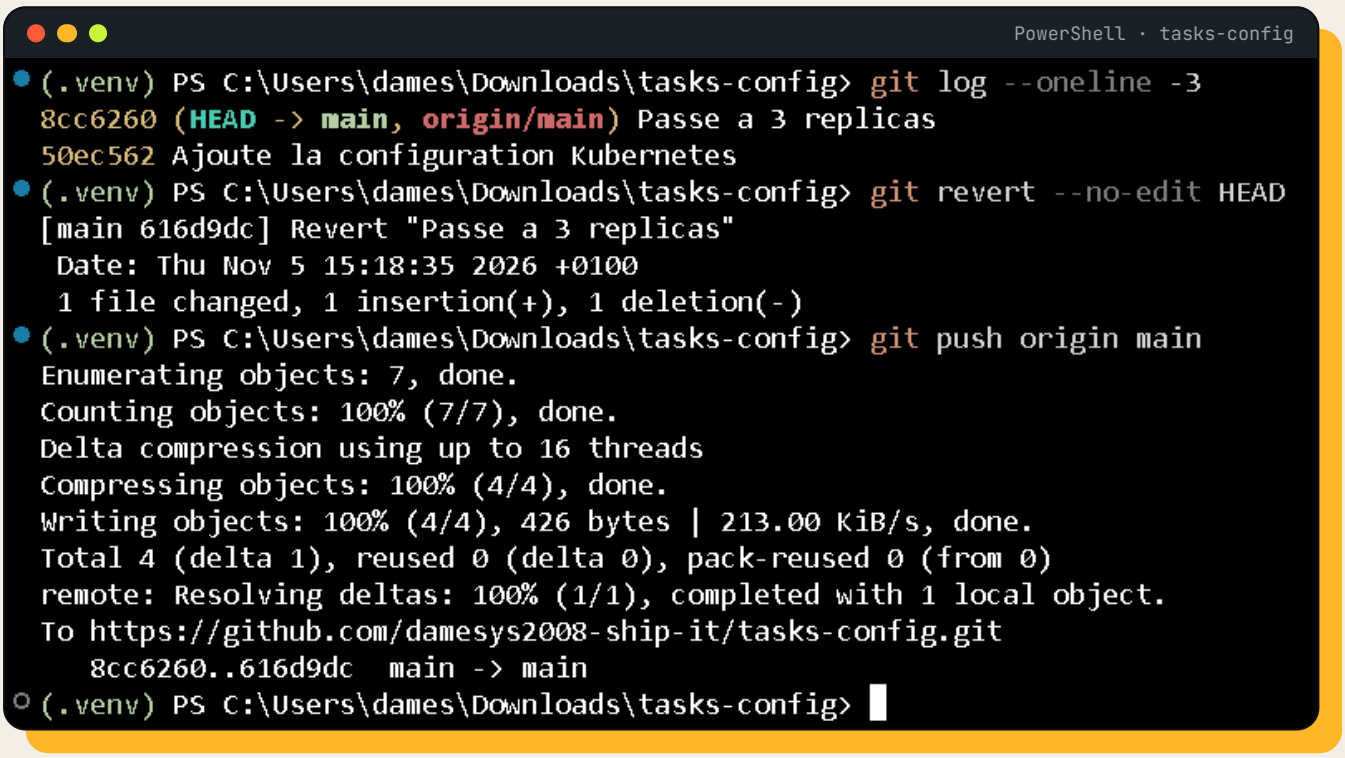
OBSERVATION ÉCRITE · PAS DE CAPTURE DEMANDÉE

Après le passage manuel à **cinq réplicas**, j'ai vu apparaître des pods supplémentaires. Argo CD a ensuite rétabli le nombre déclaré dans Git, soit **trois pods**, pendant la période d'observation.

REVERT

TP 4-7 Marche arrière, proprement

L'historique de `tasks-config`, le `git revert` du commit « Passe a 3 replicas », puis le `push` sur `main`.

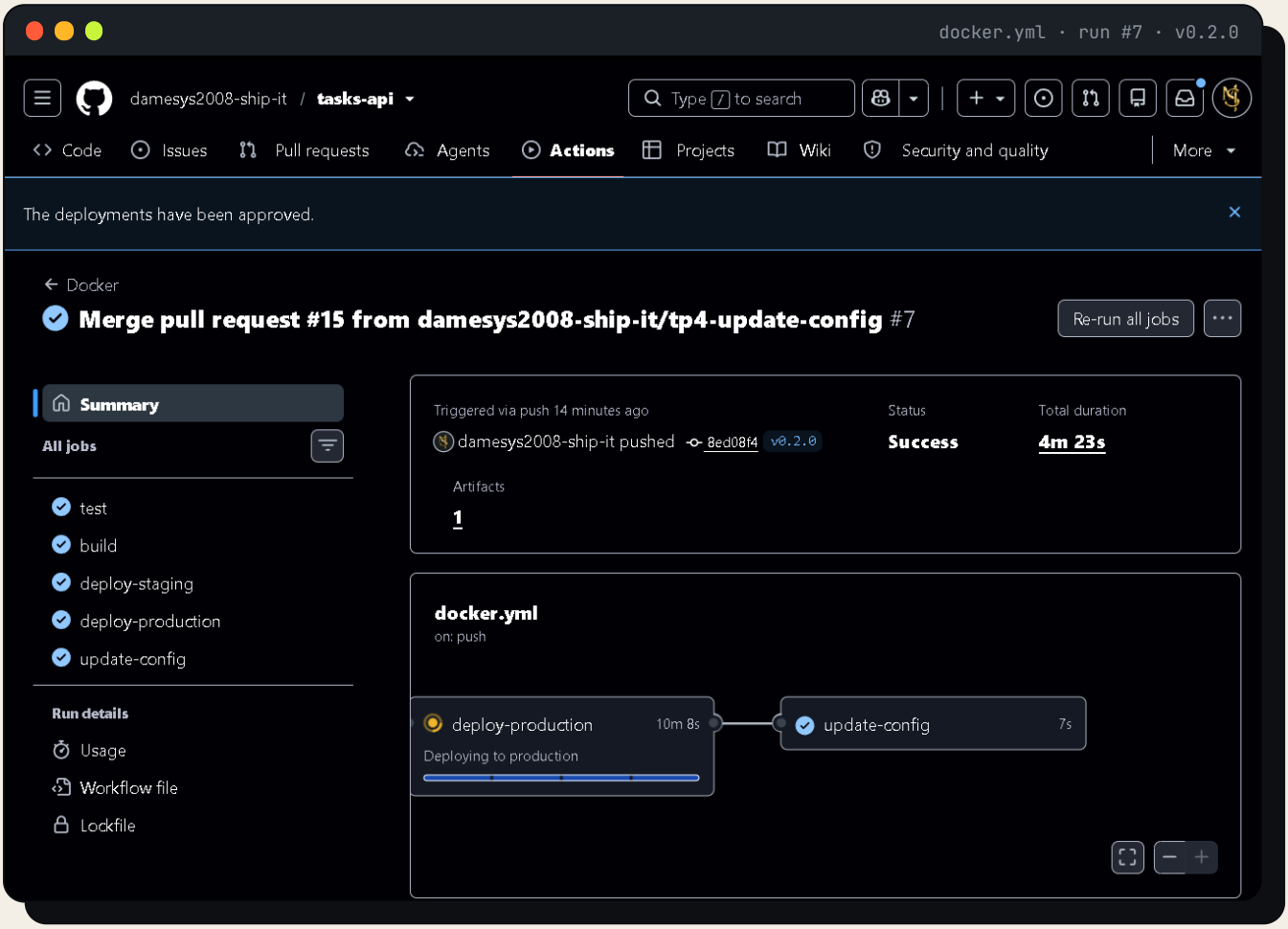


→ Revert "Passe a 3 replicas"

✓ 5 JOBS VERTS

TP 4-8 — Cinq jobs, zéro commande à la main

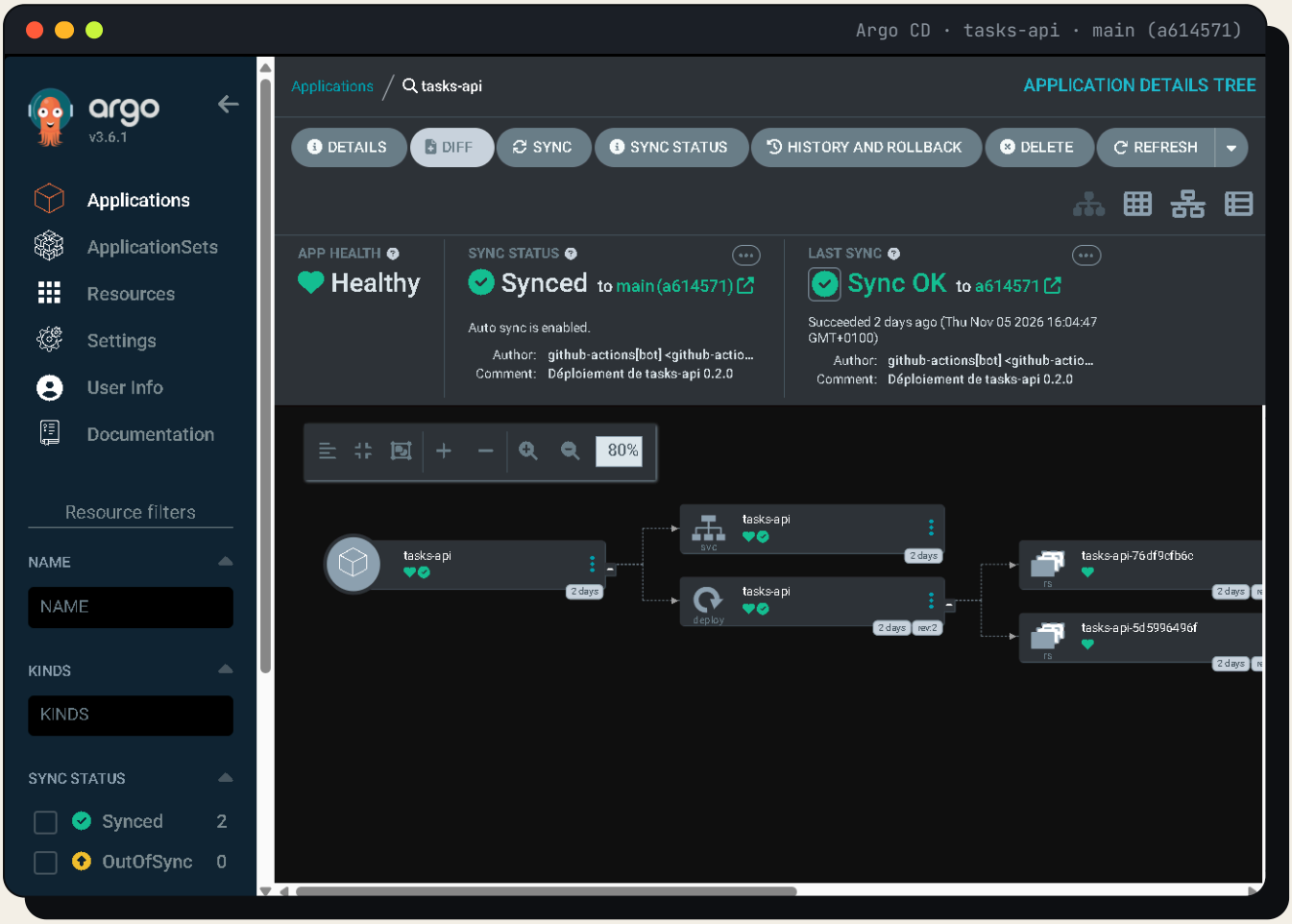
Le run complet du tag `v0.2.0` avec tous les jobs au vert, `update-config` compris.



SYNCED 0.2.0

TP 4-9 — Le bot commite, Argo CD suit

Argo CD synchronisé sur le commit automatique « Déploiement de tasks-api 0.2.0 » poussé par `github-actions[bot]` dans `tasks-config`.

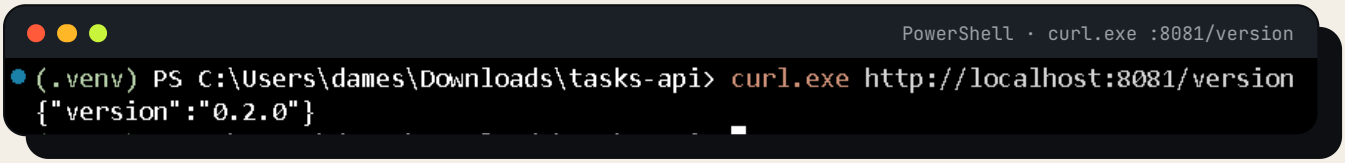


TP 4-10

✓ 0.2.0 LIVE

0.2.0, livrée par Git

`/version` répond `0.2.0` : la nouvelle version tourne dans le cluster.



→ {"version":"0.2.0"}

$ cat tp4/reponses.md

Réponses
aux questions — TP4

PARTIE 1 — KUBERNETES ET ARGO CD

01 Qu'est-ce qu'un pod ? Un Deployment ? Pourquoi un namespace ?

Un **pod** est la plus petite unité que Kubernetes déploie. Il contient un ou plusieurs conteneurs qui partagent le même environnement réseau et, au besoin, des volumes.

Un **Deployment** décrit l'état souhaité d'une application, notamment le nombre de réplicas. Kubernetes crée et remplace les pods pour maintenir cet état.

Un **namespace** sépare logiquement les ressources d'un cluster. Il facilite leur organisation et permet de limiter les accès à un espace donné.

PARTIE 2 — SÉPARATION DES DÉPÔTS

02 Pourquoi séparer le dépôt de l'application et celui de configuration ?

Le dépôt `tasks-api` contient le code et **déclenche la CI**, qui teste l'application et publie son image. Le dépôt `tasks-config` décrit **l'état attendu dans Kubernetes**. Argo CD surveille ce dépôt et applique ses changements au cluster. Cette séparation clarifie les responsabilités, facilite l'audit et permet de contrôler distinctement les droits d'accès et les déploiements.

PARTIE 3 — RÉCONCILIATION ET RETOUR ARRIÈRE

03 Que signifie `selfHeal` ? Pourquoi le retour arrière est-il simple en GitOps ?

`selfHeal` demande à Argo CD de **corriger les modifications faites directement dans le cluster** afin de le faire correspondre à la configuration déclarée dans Git. Dans l'exercice, le nombre de pods est brièvement monté à cinq, puis Argo CD l'a ramené à trois, conformément au dépôt.

Le retour arrière est simple parce que la **configuration est versionnée dans Git**. Un `git revert` crée un nouveau commit qui annule le changement précédent sans effacer l'historique ; Argo CD lit ce nouvel état et réconcilie le cluster avec lui.

PARTIE 4 — DU TAG AU POD

04 Résumer en cinq lignes le chemin d'un `git tag` jusqu'au pod.

01 GIT TAG

Le tag `v0.2.0` poussé sur `tasks-api` déclenche le workflow GitHub Actions.

02 PIPELINE

Le workflow exécute les tests, construit et publie l'image, puis passe par les déploiements staging et production (avec approbation).

03 UPDATE-CONFIG

Après la production, `update-config` enlève le préfixe `v`, met à jour le manifeste de `tasks-config`, puis pousse un commit.

04 ARGO CD

Argo CD tire la configuration de Git et synchronise le cluster avec le manifeste qui référence l'image `0.2.0`.

05 PODS

Kubernetes remplace les anciens pods ; les nouveaux conteneurs tirent l'image publiée et servent la version `0.2.0`.

05 Qui pousse et qui tire ? Quelle différence avec un déploiement CI/CD classique ?

GitHub Actions pousse l'image dans GHCR et le commit de configuration dans `tasks-config`. **Argo CD tire** la configuration depuis Git ; ensuite, les nœuds Kubernetes tirent l'image depuis GHCR.

Dans un déploiement CI/CD classique, le pipeline **pousse souvent directement** les changements vers le cluster, par exemple avec une commande de déploiement. En GitOps, le pipeline met à jour Git et Argo CD, qui surveille le dépôt, **tire l'état souhaité** et l'applique au cluster.

✓ RÉSULTAT VÉRIFIÉ

Argo CD affichait l'application **Healthy** et **Synced** sur le commit de déploiement, et l'API répondait `{"version":"0.2.0"}`.

TP4 — git tag → pod, sans commande manuelle · `exit 0`